# A Note on Scaling Laws for Transformer Models — Part I

Accounting and Empirical Scaling Laws

Junke Wang

**Abstract**

This first note develops the quantities needed to measure neural scaling laws. It begins with parameter and compute accounting for dense Transformers, then follows the Kaplan and Chinchilla studies to separate parameter, data, and compute-optimal scaling. It closes with optimal and critical batch sizes, equal-loss data efficiency, and a practical guide to scaling-ladder design. A companion Part II studies parametrizations for transferring training recipes across width and depth, and how the learning rate changes with training length.

## Contents

**1 Introduction** **3**

1.1 Why fit scaling laws? . . . 3

1.2 Nominal scale versus effective scale . . . 3

**2 Transformer Accounting and Global Conventions** **4**

2.1 Architecture and notation . . . 4

2.2 Parameter count . . . 4

2.3 Forward and training compute . . . 6

2.4 A small mathematical toolbox . . . 8

**3 The Kaplan Scaling Laws** **9**

3.1 Choosing a scale variable: shape and embeddings . . . 9

3.2 From a scale variable to power laws and transfer . . . 11

3.3 Joint dependence on model size and data . . . 14

3.3.1 The two edge laws . . . 14

3.3.2 Constructing the joint law . . . 14

3.3.3 How the fit is run and what the result shows . . . 15

3.4 Critical batch size . . . 18

3.5 Fixed compute: how large should the model be? . . . 22

3.6 What Kaplan has established . . . 26

**4 Chinchilla: Measuring the Allocation Directly** **28**

4.1 What changes from Section 3.5 . . . . . 28

4.2 Three ways to measure the allocation . . . . . 28

**5 From Scaling Laws to Scaling Experiments** **30**

5.1 $B_{\text{opt}}$ and $B_{\text{crit}}$: practical choices . . . . . 30

5.2 Improving critical-batch estimation . . . . . 32

5.3 Designing a scaling ladder . . . . . 33

# 1 Introduction

Suppose we can afford exactly one very large training run. How large should the model be, how many tokens should it see, and what loss will it reach? Running the experiment to find out is precisely what we cannot afford. Scaling laws answer these questions by extrapolating from runs small enough to repeat.

## 1.1 Why fit scaling laws?

Larger models, more data, and more compute tend to lower the loss. This says that scale helps, but not by how much. The power-law hypothesis makes the trend quantitative: for a scale variable $x$,

$$\mathcal{L}(x) \approx A\, x^{-\alpha}, \tag{1}$$

or, taking logarithms of both sides,

$$\log \mathcal{L}(x) \approx \log A - \alpha \log x. \tag{2}$$

On log–log axes the law is a straight line with slope $-\alpha$: multiplying $x$ by a factor $k$ multiplies the loss by $k^{-\alpha}$, wherever on the curve we start.

The constants $A$ and $\alpha$ are estimated from *pilot runs*: a ladder of complete but cheap training runs that differ in one scale variable while everything else is held fixed. Depending on the question, $x$ can be the number of parameters $N$, the number of training tokens $D$, the training compute $C$ in FLOPs, or the number of optimization steps. Each run contributes one point $(x_k, \mathcal{L}_k)$. Fitting the line in Eq. (2) through these points predicts what a run ten or a hundred times larger should reach. Points that bend away from the line are informative too: they signal that something other than $x$ has become the bottleneck.

## 1.2 Nominal scale versus effective scale

Even a clean fit leaves two deeper questions open.

1. **Is scaling parameters the same as scaling effective parameters?** $N$ counts weights, not the capacity the model actually uses. Added parameters may be rarely used, may barely move away from their random initialization, or may learn redundant and meaningless representations.

2. **Is scaling tokens the same as scaling effective data?** $D$ counts tokens, not information. Repeated data can still improve the model, but its marginal value need not match that of fresh data; excessive reuse can lead to memorization and overfitting. The effect depends on the task, repetition count, augmentation, and optimization protocol. Diffusion training also resamples noise and timesteps when an image is reused. Noisy data can dilute useful signal or change the distribution being learned.

"Effective scale" is an intuition here, not an independently measured scalar that uniquely determines the loss. Nominal counts are useful empirical coordinates when model utilization and data quality remain comparable along the ladder. Under a simple constant-fraction model, a fixed difference in effective scale is absorbed into the fitted amplitude; a changing fraction can alter the apparent exponent. More generally, changing data quality, repetition, or optimization can invalidate an extrapolation without changing the nominal counts.

**Reading route.** Start with the counting conventions in Section 2. For the practical question—how to turn a compute budget into a model size, batch, step count, and predicted loss curve—follow Section 3.5, then Section 4 for the revised allocation. The earlier Kaplan subsections explain where the fitted laws come from; their detailed plot readings can be revisited after the main calculation. Section 5 compares later results and distinguishes the optimal batch from the critical batch. Part II explains how to train the different model sizes comparably.

# 2 Transformer Accounting and Global Conventions

## 2.1 Architecture and notation

Consider a dense Transformer with $L$ identical blocks. "Dense" means that every token executes the same attention and feed-forward parameters; it is used here to distinguish the model from a sparsely activated MoE. We use SwiGLU and RMSNorm as the reference block. DiTs may instead use GELU, LayerNorm, adaptive normalization, or cross-attention; those architecture-specific additions are outside the reference model counted in this section. Biases are omitted because they are lower order in parameter count and compute.

The architectural dimensions are

| Symbol | Definition |
|---|---|
| $L$ | Number of Transformer blocks |
| $d_{\text{model}}$ | Width of the residual stream |
| $n_{\text{head}}$ | Number of attention heads |
| $d_{\text{head}}$ | Hidden width of one attention head |
| $d_{\text{attn}} = n_{\text{head}} d_{\text{head}}$ | Total concatenated attention hidden width |
| $d_{\text{ff}}$ | Intermediate hidden width of the SwiGLU FFN |
| $V$ | Vocabulary size |
| $d_{\text{vae}}$ | Channel dimension of the VAE latent consumed by a DiT |
| $p_{\text{DiT}}$ | DiT patch size applied on the VAE latent grid, often 2 |
| $d_{\text{io}}$ | Modality-specific input/output interface width |
| $m$ | Number of language or image tokens in one input sequence |
| $B$ | Number of tokens in one parameter update |
| $S$ | Number of parameter updates |
| $E_{\text{tok}} = BS$ | Total number of token exposures in a training run |
| $A_{\text{mask}}$ | Valid query–key pairs among the $m$ tokens, per head |

## 2.2 Parameter count

**Input and output interfaces** We assume untied input and output parameters. For an LLM, the token embedding and vocabulary head therefore contain

$$N_{\text{in}}^{\text{LLM}} = V d_{\text{model}}, \qquad N_{\text{out}}^{\text{LLM}} = V d_{\text{model}}. \tag{3}$$

We assume RoPE throughout, so positional encoding adds no learned parameters.

A DiT receives continuous VAE latents rather than vocabulary indices. The VAE's internal pixel-to-latent mapping belongs to the VAE and is not part of the DiT parameter count. Let $d_{\text{vae}}$ be the channel dimension of the VAE latent. The DiT then patchifies its latent grid with its own patch size $p_{\text{DiT}}$, so one flattened DiT token has dimension $p_{\text{DiT}}^2 d_{\text{vae}}$. Assuming that the

DiT predicts a latent patch of the same dimension, its input and output projections contain

$$N_{\text{in}}^{\text{DiT}} = p_{\text{DiT}}^2 d_{\text{vae}} d_{\text{model}}, \qquad N_{\text{out}}^{\text{DiT}} = d_{\text{model}} p_{\text{DiT}}^2 d_{\text{vae}}.$$

The latent-patch width is algebraically analogous to $V$ in the interface parameter count, but it is a continuous feature dimension, not a vocabulary. Define

$$d_{\text{io}} := \begin{cases} V, & \text{for the reference LLM,} \\ p_{\text{DiT}}^2 d_{\text{vae}}, & \text{for the reference DiT.} \end{cases} \tag{4}$$

The modality-specific interface count is then

$$N_{\text{io}} := N_{\text{in}} + N_{\text{out}} = 2 d_{\text{io}} d_{\text{model}}. \tag{5}$$

**Attention in one block** For $X \in \mathbb{R}^{m \times d_{\text{model}}}$, ordinary multi-head attention uses

$$W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{attn}}}, \qquad W_O \in \mathbb{R}^{d_{\text{attn}} \times d_{\text{model}}}. \tag{6}$$

Therefore,

$$N_{\text{attn/block}} = 4 d_{\text{model}} d_{\text{attn}}. \tag{7}$$

**SwiGLU in one block** The gate and up projections map from $d_{\text{model}}$ to $d_{\text{ff}}$, and the down projection maps back:

$$\text{SwiGLU}(\boldsymbol{x}) = W_d(\text{SiLU}(W_g \boldsymbol{x}) \odot W_u \boldsymbol{x}). \tag{8}$$

The three weight matrices contain

$$N_{\text{ff/block}} = 3 d_{\text{model}} d_{\text{ff}} \tag{9}$$

parameters. By comparison, a non-gated two-matrix FFN has $2 d_{\text{model}} d_{\text{ff}}$ parameters at the same intermediate width.

**Normalization and the complete model** A pre-norm block has two RMSNorm gain vectors and therefore $2 d_{\text{model}}$ normalization parameters. A final RMSNorm contributes another $d_{\text{model}}$. The detailed parameter decomposition is

$$\boxed{\begin{aligned} N_{\text{params}} = & \underbrace{N_{\text{in}} + N_{\text{out}}}_{\text{input/output embeddings or projections}} \\ & + \underbrace{L \left( \underbrace{4 d_{\text{model}} d_{\text{attn}}}_{\text{attention}} + \underbrace{3 d_{\text{model}} d_{\text{ff}}}_{\text{SwiGLU FFN}} + \underbrace{2 d_{\text{model}}}_{\text{RMSNorm}} \right)}_{\text{repeated Transformer blocks}} \\ & + \underbrace{d_{\text{model}}}_{\text{final RMSNorm}}. \end{aligned}} \tag{10}$$

Combining like terms gives

$$\boxed{N_{\text{params}} = N_{\text{io}} + L d_{\text{model}} (4 d_{\text{attn}} + 3 d_{\text{ff}} + 2) + d_{\text{model}}.} \tag{11}$$

When the normalization terms are lower order,

$$N_{\text{params}} \approx N_{\text{io}} + L d_{\text{model}} (4 d_{\text{attn}} + 3 d_{\text{ff}}). \tag{12}$$

### 2.3 Forward and training compute

This subsection counts compute inside the repeated Transformer blocks only. It does not include the embedding lookup, vocabulary projection, DiT input/output patch projections, optimizer updates, communication, or hardware-specific kernel overhead. Those interface terms can be added separately, but the scaling coordinate used below follows the common non-embedding convention.

The basic counting rule is simple: if $X \in \mathbb{R}^{m \times d_{\text{in}}}$ is multiplied by $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$, each of the $md_{\text{out}}$ output entries is a dot product of length $d_{\text{in}}$. Hence

$$\text{MACs}(XW) = md_{\text{in}}d_{\text{out}}, \qquad \text{FLOPs}(XW) \approx 2md_{\text{in}}d_{\text{out}}, \tag{13}$$

where one multiply–accumulate (MAC) is counted as two floating-point operations (FLOPs): one multiplication and one addition. Throughout this note, every dimensional compute symbol $C$ counts FLOPs, not MACs; published values may be quoted in PF-days, with 1 PF-day $= 8.64 \times 10^{19}$ FLOPs. We use lowercase $c_{\min}$ for the numerical compute in PF-days when inserting the historical Kaplan fit coefficients. This convention is why each matrix-multiplication term below carries a factor of 2. Operations that are not multiply–accumulates, such as an exponential or a division, are counted as one FLOP each.

**Attention forward pass** For one block, ignoring residual additions for the moment, consider self-attention over one sequence of $m$ tokens: the queries, keys, and values all come from the same input $X$. Attention must be computed separately for each head. After the dense projections, reshape $Q, K, V$ into head-indexed tensors and write

$$\begin{aligned}
&\textbf{Input:} && X \in \mathbb{R}^{m \times d_{\text{model}}} \\
&1. && Q = XW_Q, \quad K = XW_K, \quad V = XW_V \\
&2. && A_h = Q_h K_h^\top / \sqrt{d_{\text{head}}}, \quad h = 1, \dots, n_{\text{head}} \\
&3. && P_h = \text{softmax}(A_h) \quad \text{with the attention mask applied} \\
&4. && H_h = P_h V_h \\
&5. && Y = \text{Concat}(H_1, \dots, H_{n_{\text{head}}}) W_O.
\end{aligned}$$

Each score matrix $A_h$ is $m \times m$: one row per query token and one column per key token. Let $A_{\text{mask}}$ be the number of its entries that the attention mask allows:

$$A_{\text{mask}} = \begin{cases} m^2 & \text{bidirectional attention,} \\ \dfrac{m(m+1)}{2} \approx \dfrac{m^2}{2} & \text{causal attention.} \end{cases} \tag{14}$$

In the causal case, query $t$ may attend only to keys $1, \dots, t$, so the count is $1 + 2 + \cdots + m$. Every head has its own score matrix, so all heads together hold $n_{\text{head}} A_{\text{mask}}$ valid entries. The attention FLOPs split into two groups:

$$\left.\begin{aligned} C_{QKV} &\approx 2m \cdot 3d_{\text{model}}d_{\text{attn}} \\ C_O &\approx 2md_{\text{attn}}d_{\text{model}} \end{aligned}\right\} \text{ counted in } C\text{: learned projections,}$$

$$\left.\begin{aligned} C_{QK^\top} &\approx 2A_{\text{mask}}d_{\text{attn}} \\ C_{\text{softmax}} &\approx 5n_{\text{head}}A_{\text{mask}} \\ C_{PV} &\approx 2A_{\text{mask}}d_{\text{attn}} \end{aligned}\right\} \text{ not counted: parameter-free.} \tag{15}$$

The first group scales with the parameter count; the second scales with $A_{\text{mask}} \propto m^2$ and is left out of the scaling coordinate below. The factor 5 in $C_{\text{softmax}}$ counts the max, subtraction,

exponential, sum, and division applied to each score entry. These operations still consume compute despite having no learned parameters. Their omission is justified only when projections dominate. The causal count assumes a kernel that skips masked pairs; a dense implementation that computes the full score matrix before masking can do more work. Long contexts and small models therefore require a fuller FLOP estimate rather than automatic use of $6NE_{\text{tok}}$.

**SwiGLU FFN forward pass** The reference FFN uses three dense projections.

$$\begin{aligned} &\textbf{Input:} && X \in \mathbb{R}^{m \times d_{\text{model}}} \\ &1. && G = XW_g, \quad U = XW_u \\ &2. && Z = \text{SiLU}(G) \odot U \\ &3. && Y = ZW_d. \end{aligned}$$

The FFN FLOPs are

$$\begin{aligned} C_{g,u} &\approx 2m \cdot 2d_{\text{model}} d_{\text{ff}}, \\ C_{\text{SiLU}+\odot} &= O(md_{\text{ff}}), \\ C_d &\approx 2md_{\text{ff}} d_{\text{model}}. \end{aligned}$$

The matrix multiplications are $XW_g$, $XW_u$, and $ZW_d$. The SiLU and elementwise product add lower-order FLOPs compared with the dense projections when $d_{\text{ff}}$ is large.

**Normalization and residual terms** RMSNorm also costs FLOPs. Per token, it computes a mean square, a reciprocal square root, and a rescaling, so one RMSNorm is $O(d_{\text{model}})$ per token. A pre-norm block has two such normalizations, plus residual additions of the same order. These terms matter for exact profiling, especially in small models, but they are not the dominant term in the projection-dominated scaling coordinate.

**Overall approximation** The learned block matrices contain

$$N_{\text{core}} := Ld_{\text{model}} \left(4d_{\text{attn}} + 3d_{\text{ff}}\right). \tag{16}$$

Counting one multiply–accumulate as two FLOPs, the dominant forward cost can be written directly as

$$C_{\text{forward}}(m) \approx \underbrace{2mN_{\text{core}}}_{\text{repeated block projections}}, \tag{17}$$

where lower-order normalization, nonlinearities, softmax, and residual operations are omitted. The parameter-free attention-pair computation is also omitted from this leading-order expression; it can become important at long context lengths.

**From forward to training compute** For a linear layer $Y = XW$ with incoming gradient $G = \partial\mathcal{L}/\partial Y$, the backward pass computes

$$\frac{\partial \mathcal{L}}{\partial X} = GW^\top, \qquad \frac{\partial \mathcal{L}}{\partial W} = X^\top G, \tag{18}$$

two matrix multiplications of the same size as the forward $XW$. Hence $C_{\text{train}} \approx 3\, C_{\text{forward}}$, and over $E_{\text{tok}} = BS$ training tokens,

$$\boxed{C_{\text{train}} \approx 3 \cdot 2N_{\text{core}} E_{\text{tok}} = 6N_{\text{core}} BS.} \tag{19}$$

## 2.4 A small mathematical toolbox

Only a few algebra and calculus rules are needed for the rest of this note.

**Powers, logarithms, and exponentials** For positive numbers $a, b$ and any real number $c$,

$$\log(ab) = \log a + \log b, \qquad \log\left(\frac{a}{b}\right) = \log a - \log b, \qquad \log(a^c) = c \log a. \tag{20}$$

The exponential $\exp(x) = e^x$ reverses the natural logarithm:

$$\exp(\log a) = a, \qquad \log(\exp x) = x, \qquad \exp(x + y) = \exp(x)\exp(y). \tag{21}$$

These rules turn multiplication and powers into addition. For example, a power law becomes a straight line after taking logs:

$$\mathcal{L}(x) = Ax^{-\alpha} \quad \Longrightarrow \quad \log \mathcal{L}(x) = \log A - \alpha \log x. \tag{22}$$

Thus, if $v = \log \mathcal{L}$ and $u = \log x$, a fitted line $v = su + b$ has $\alpha = -s$ and $A = \exp(b)$. The logarithm is defined here only for positive quantities, which is satisfied by losses, parameter counts, tokens, and compute.

**Slopes and derivatives** The derivative $f'(x)$ is the local slope of a curve: it tells us how much $f(x)$ changes after a small change in $x$. Three rules used repeatedly are

$$\frac{d}{dx}x^p = px^{p-1}, \qquad \frac{d}{dx}\log x = \frac{1}{x}, \qquad \frac{d}{dx}\exp x = \exp x. \tag{23}$$

For a function of several variables, $\partial f/\partial x$ means changing $x$ while holding the other variables fixed. At a smooth interior minimum, the derivative in every allowed direction is zero.

**Taylor expansion** The derivative at $x_0$ is defined by the change in the output divided by the change in the input, as that input change approaches zero:

$$f'(x_0) := \lim_{h \to 0} \frac{f(x_0 + h) - f(x_0)}{h}. \tag{24}$$

For a small but nonzero $h$, the fraction is approximately $f'(x_0)$. Multiplying by $h$ gives the first-order approximation $f(x_0 + h) - f(x_0) \approx f'(x_0)h$. Taylor expansion improves it by adding the curvature and higher-order corrections:

$$f(x_0 + h) - f(x_0) = f'(x_0)h + \frac{f''(x_0)}{2!}h^2 + \frac{f^{(3)}(x_0)}{3!}h^3 + \frac{f^{(4)}(x_0)}{4!}h^4 + O(h^5). \tag{25}$$

Here $f^{(3)}$ and $f^{(4)}$ are the third and fourth derivatives, while $k! = 1 \cdot 2 \cdots k$. The pattern is that the $k$-th derivative is multiplied by $h^k/k!$. When $h$ is small, each successive power of $h$ is smaller, so the first one or two terms often suffice. A Taylor expansion is local: it need not describe the function when $h$ is large.

The most common special case expands around zero. Set $x_0 = 0$ and rename $h$ as $x$ in Eq. (25):

$$f(x) - f(0) = f'(0)x + \frac{f''(0)}{2}x^2 + O(x^3). \tag{26}$$

This is the Taylor expansion around zero, also called the Maclaurin expansion.

# 3 The Kaplan Scaling Laws

This section reviews Sections 3–6 of Kaplan et al. [8]. We reproduce their principal equations and empirical fits, but first separate their experimental conventions from those of Section 2.

**Different conventions** Kaplan et al. study decoder-only autoregressive language models on WebText2, using token-level cross entropy in nats. Their model-size variable excludes vocabulary and positional embeddings, their input and output embeddings are tied, and their FFN has two linear maps rather than the three-map gated SwiGLU used in Section 2. Their compute estimate also retains only the non-embedding matrix multiplications. We therefore write

$$N_{\mathrm{K}} := \text{Kaplan non-embedding parameter count}, \qquad C_{\mathrm{K}} := 6N_{\mathrm{K}}BS, \tag{27}$$

where $B$ is the number of tokens per parameter update and $S$ is the number of updates. The factor 6 is exactly the projection-dominated approximation derived in Section 2.3: roughly $2N_{\mathrm{K}}$ forward FLOPs and $4N_{\mathrm{K}}$ backward FLOPs per token. It is a leading-order coordinate, not an exact hardware count; it omits embedding/de-embedding operations, the quadratic attention-pair computation, normalization, nonlinearities, optimizer updates, and communication.

In the notation of Section 2, $N_{\text{params}}$ includes untied input and output interfaces. The detailed forward decomposition lists interface and attention-pair compute, but the leading coordinate in Eq. (17) omits both. Importantly, whether the input and output interfaces are tied does not change the horizontal variables of Kaplan et al.'s scaling laws: both $N_{\mathrm{K}}$ and $C_{\mathrm{K}}$ exclude the embedding/de-embedding interfaces either way. Weight tying is nevertheless an architectural condition under which their losses were measured, so changing it could still affect the empirical loss and fitted constants even though it does not affect their counting convention. Consequently, the numerical constants below remain conditional on their architecture, dataset, and counting convention, and should be re-fitted before being combined with $N_{\text{params}}$ or full-forward-compute formulas.

The same warning is stronger for DiTs. The functional forms may be useful hypotheses for diffusion training, but cross-entropy loss, tokenization, WebText2, and the reported fitted constants are LLM-specific. We use $\mathcal{L}$ for loss below so that $L$ remains the number of Transformer blocks defined in Section 2.

## 3.1 Choosing a scale variable: shape and embeddings

Before fitting any power law, Kaplan et al. ask which coordinate best measures model size. They vary one shape dimension at a time at approximately fixed $N_{\mathrm{K}}$, and the vertical axis in Figure 1 is the percentage loss increase over the model-size baseline, not the absolute loss [8].

The three panels of Figure 1 carry one message: within a wide range, shape matters weakly compared with $N_{\mathrm{K}}$.

- **FFN ratio (left).** $\mathcal{L}$ has a broad minimum near $d_{\text{ff}}/d_{\text{model}} \sim 1$; only very large FFNs hurt, because they force the rest of the model to be too narrow.
- **Depth–width (middle).** A roughly $40\times$ change in $d_{\text{model}}/L$ costs only a few percent, so there is a broad basin rather than one optimal aspect ratio.
- **Head dimension (right).** $\mathcal{L}$ is flat over a wide range of $d_{\text{model}}/n_{\text{head}}$, so head partitioning matters little.

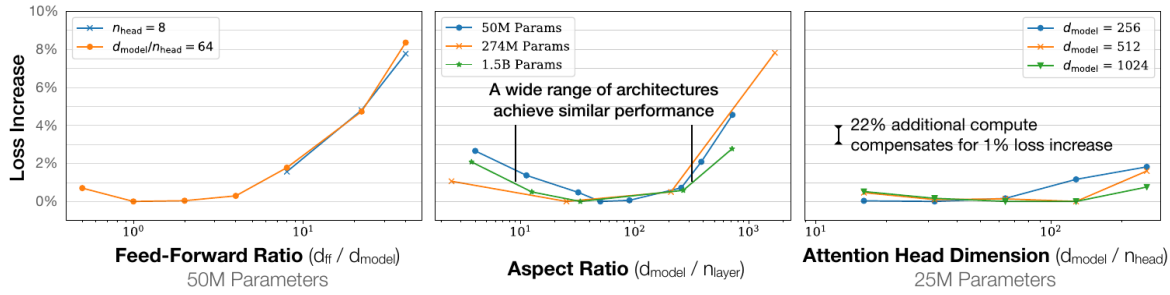



Figure 1: Controlled shape sweeps at fixed non-embedding parameter count. The three panels vary the FFN ratio, depth-to-width aspect ratio, and attention-head dimension. They report the increase in loss relative to the model-size baseline. Reproduced from Figure 5 of Kaplan et al. [8].

Figure 2 then fixes what "model size" means. Split the total count into block and embedding parameters,

$$N_{\text{total}} = N_{\text{K}} + N_{\text{embed}}, \qquad N_{\text{embed}} \approx (V + n_{\text{ctx}})\, d_{\text{model}}, \tag{28}$$

where the embedding term is a lookup interface whose share $N_{\text{embed}}/N_{\text{total}}$ grows for wide/shallow models and shrinks for deep/narrow ones. On the left, indexing by $N_{\text{total}}$ mixes this varying overhead into the coordinate, so models with the same $N_{\text{total}}$ but different depth land at different losses and the curves split. On the right, indexing by $N_{\text{K}}$ removes the depth-dependent overhead, and the same models collapse onto one trend (except one-layer models and extreme aspect ratios).

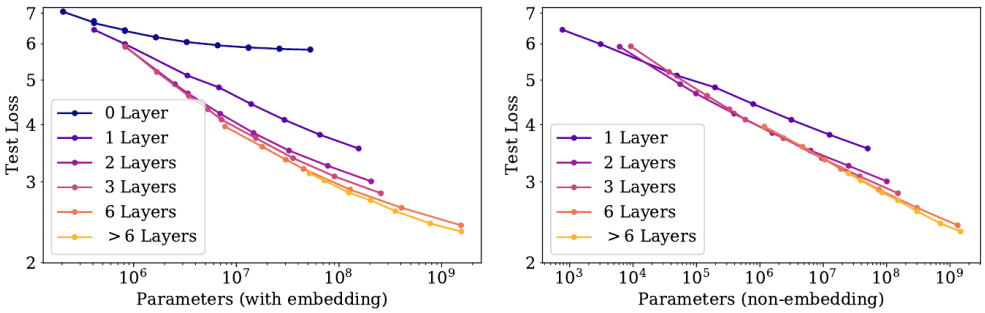



Figure 2: The same family of models indexed by total parameters (left) and non-embedding parameters (right). Including embeddings separates the curves by depth; excluding them collapses models with at least two layers and non-extreme shapes onto a common trend. This establishes a shared scale variable, not by itself a power law. Reproduced from Figure 6 of Kaplan et al. [8].

This collapse answers only the *choice-of-coordinate* question. Each color represents a different depth, and the points along one colored curve vary the width and hence the model size. Their overlap on the right says that, after excluding embeddings, depth contributes little additional information: to first approximation one may write $\mathcal{L} \approx f(N_{\text{K}})$ with the same function $f$ for all these depths. As modern LLMs continue to scale, embedding and vocabulary-interface parameters can become large again, and recent work suggests that vocabulary and embedding capacity can itself affect loss [7, 14]. The scale coordinate must therefore be revalidated rather than assuming that Kaplan's non-embedding collapse transfers unchanged.

### 3.2 From a scale variable to power laws and transfer

Figure 2 establishes that $N_{\mathrm{K}}$ is a useful model-size coordinate, but it does not determine how loss depends on that coordinate. Kaplan et al. next test a stronger empirical claim: when the other resources are sufficiently non-binding, loss is well described by a power law in the resource that remains limiting. They test this separately for non-embedding model size $N_{\mathrm{K}}$, distinct training data $D$, and training compute $C$.

Write the limiting resource generically as $x$. The one-dimensional hypothesis has the common form

$$\begin{aligned} \mathcal{L}(x) &\approx Ax^{-\alpha} = \left(\frac{x_c}{x}\right)^{\alpha}, \qquad A > 0, \quad \alpha > 0, \\ \log \mathcal{L}(x) &\approx \log A - \alpha \log x = \alpha \log x_c - \alpha \log x. \end{aligned} \tag{29}$$

where $x$ will successively denote $N_{\mathrm{K}}$, $D$, or $C$, and each case has its own exponent and scale. This form is not derived from the Transformer. Earlier learning-curve and neural-scaling studies had reported approximate power laws [5, 13], while Kaplan et al.'s own sweeps showed smooth improvement over several orders of magnitude without an evident characteristic scale. A power law is therefore a natural low-parameter hypothesis: multiplying the resource by a fixed factor predicts a fixed fractional reduction in loss. The authors also tested alternatives such as logarithmic fits and found the power law qualitatively more accurate [8]. It remains an empirical fit over the observed regime; an irreducible loss floor must eventually bend the trend.

Figure 3 summarizes the three experiments. The $N_{\mathrm{K}}$ and $D$ panels vary one resource while keeping the other bottlenecks loose. The compute panel instead combines checkpoints from many model sizes, so we discuss dataset size first, then model size, and finally the constructed compute envelope. In every case the power law is supported by agreement over the measured range, not derived from the Transformer architecture.

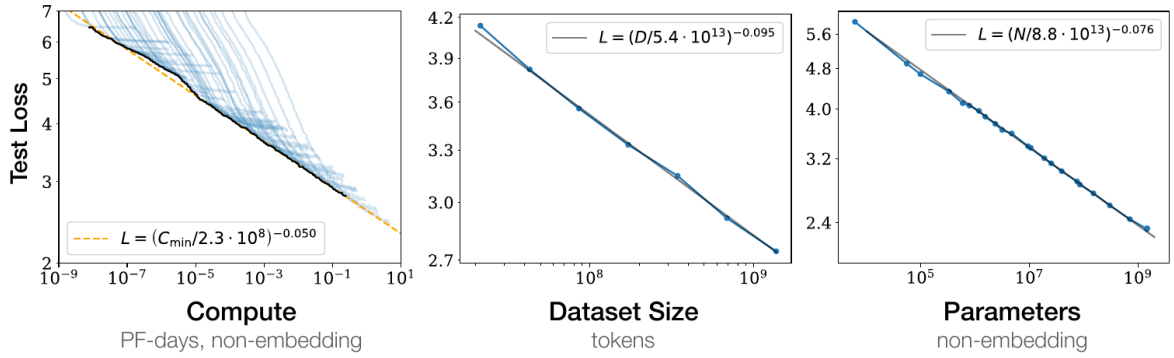



Figure 3: The three one-dimensional scaling experiments, displayed left-to-right as compute, dataset size, and non-embedding model size. The horizontal and loss axes use logarithmic scales; the loss ticks are labelled with loss values rather than their logarithms. A power law is a straight line in these log–log coordinates. The dataset-size panel holds the architecture fixed at the large $(n_{\mathrm{layer}}, d_{\mathrm{model}}) = (36, 1280)$ model, while the model-size panel trains each Transformer on the full WebText2 dataset. The compute panel is instead a lower envelope over checkpoints from many model sizes. The discussion below analyzes dataset size first, then model size, and finally that constructed envelope. Reproduced from Figure 1 of Kaplan et al. [8].

**Dataset size: middle panel** Here the architecture is held fixed at the large $(n_{\mathrm{layer}}, d_{\mathrm{model}}) = (36, 1280)$ model, while each blue point uses a different fixed subset containing $D$ tokens. Every run is early-stopped when test loss ceases to improve, so the vertical value is the best test loss

attainable from that subset rather than loss at a common step or common compute budget. Again the blue segments connect observations and the gray curve is the fit

$$\begin{aligned} \mathcal{L}(D) &\approx \left(\frac{D_c}{D}\right)^{\alpha_D}, \\ \log \mathcal{L}(D) &\approx \alpha_D \log D_c - \alpha_D \log D, \\ \alpha_D &\approx 0.095, \qquad D_c \approx 5.4 \times 10^{13} \text{ tokens}. \end{aligned} \tag{30}$$

On the panel's log–log axes this relation is a straight line with slope $-\alpha_D$ in logarithmic coordinates and intercept $\alpha_D \log D_c$. The numeric loss labels do not make the vertical scale linear; equal multiplicative changes, rather than equal additive changes, occupy equal distances. The screen angle of the line also depends on the axes' aspect ratio. This panel isolates the data bottleneck by using one sufficiently capable model; it is not yet the joint $\mathcal{L}(N, D)$ experiment of Section 3.3.

**Model size: right panel** Each blue point is the near-converged test loss of a different Transformer trained on the full WebText2 dataset. The models vary in both size and shape, but the horizontal coordinate counts only non-embedding parameters $N_{\mathrm{K}}$, which is why the points from Figure 2 largely share one trend. The blue segments merely connect measured points; the thin gray curve is the fitted power law

$$\begin{aligned} \mathcal{L}(N_{\mathrm{K}}) &\approx \left(\frac{N_c}{N_{\mathrm{K}}}\right)^{\alpha_N}, \\ \log \mathcal{L}(N_{\mathrm{K}}) &\approx \alpha_N \log N_c - \alpha_N \log N_{\mathrm{K}}, \\ \alpha_N &\approx 0.076, \qquad N_c \approx 8.8 \times 10^{13}. \end{aligned} \tag{31}$$

As in the dataset panel, both axes use logarithmic scales. The slope in logarithmic coordinates is $-\alpha_N$ and the intercept is $\alpha_N \log N_c$; the rendered screen angle depends additionally on the relative axis scales. This equation records the quantitative fit—especially $\alpha_N$ and $N_c$—that will later serve as the infinite-data boundary of the joint $\mathcal{L}(N_{\mathrm{K}}, D)$ law. It is therefore more than a restatement of the visual collapse in Figure 2. This is a model-capacity experiment, not an iso-compute experiment. The runs are compared near convergence with abundant data, so larger models generally consume more total compute. The panel supports a power law in non-embedding size within the tested Transformer family; it does not show that total parameters, arbitrary architectures, or equal-FLOP runs follow the same curve.

**Training compute: left panel** This panel is less direct because it is constructed from whole learning curves. Each pale-blue trace belongs to one model size. Every checkpoint at step $S$ is placed at its estimated non-embedding compute

$$C_{\mathrm{K}} = 6 N_{\mathrm{K}} B S, \qquad S = \frac{C_{\mathrm{K}}}{6 N_{\mathrm{K}} B}. \tag{32}$$

The construction is easiest to read as a sweep over a sequence of compute budgets $C_1, C_2, \ldots$. For a fixed budget $C_i$, vary the candidate model size $N_{\mathrm{K}}$. Equation (32) then tells us which step count would fit inside that budget, $S_i(N_{\mathrm{K}}) = C_i/(6 N_{\mathrm{K}} B)$. We read the loss of each model's learning curve at its corresponding step count and use the smallest observed loss as the vertical coordinate at $C_i$. Repeating this for all $C_i$ produces the heavy black curve. Thus the black line is a lower envelope, not one model's learning curve and not a set of models all trained to convergence. At the same $C_i$, a larger model is read at fewer steps and a smaller model at more steps; as $C_i$ changes, the identity of the selected model can change.

The heavy black curve is the empirical fixed-batch envelope. Kaplan et al. report compute in petaflop-days: one PF-day means sustaining $10^{15}$ FLOPs per second for one day, or $8.64 \times 10^{19}$ total FLOPs. They report the corresponding raw-compute fit as

$$\mathcal{L}(C_{\mathrm{K}}) \approx \left( \frac{1.6 \times 10^{7}\ \mathrm{PF-days}}{C_{\mathrm{K}}} \right)^{0.057} . \tag{33}$$

The orange dashed line instead shows the batch-corrected, compute-efficient trend in $C_{\min}$. Its legend uses $2.3 \times 10^{8}$ PF-days for the scale, while the paper's summary fit reports

$$\mathcal{L}(C_{\min}) \approx \left( \frac{C_c^{\min}}{C_{\min}} \right)^{\alpha_C^{\min}} , \qquad \alpha_C^{\min} \approx 0.050, \qquad C_c^{\min} \approx 3.1 \times 10^{8}\ \mathrm{PF-days}. \tag{34}$$

The numerical difference between the legend and summary values reflects different fits in the paper, not a unit conversion. The corrected trend is developed in Sections 3.4– 3.5. It should not be identified with the raw black envelope even though both are displayed in the same panel.

**Generalization across text distributions** The models used for this test are trained only on WebText2 and evaluated on Internet Books, Books, Wikipedia, and Common Crawl. The experiment asks whether the WebText2 scaling trend is merely in-distribution memorization or tracks a more transferable improvement.

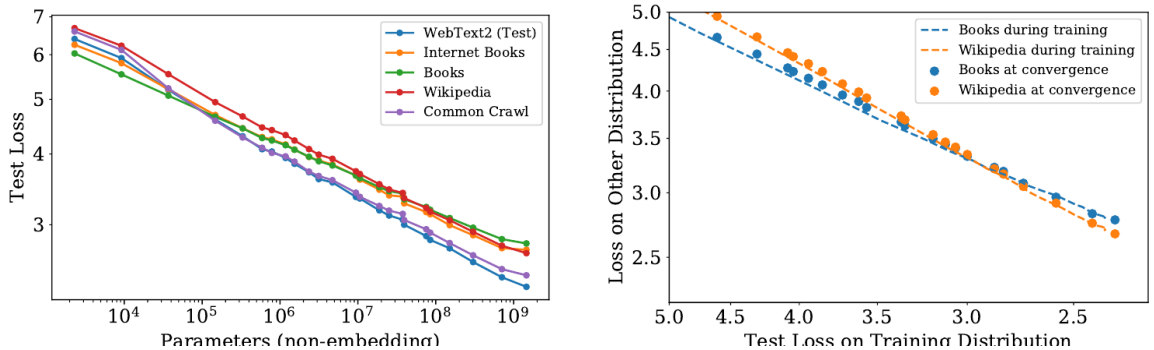



Figure 4: Generalization from WebText2 to other text distributions. Left: every evaluation distribution improves smoothly with non-embedding model size. Right: losses measured during training and at convergence lie on nearly the same mapping from WebText2 loss to target-distribution loss. Reproduced from Figure 8 of Kaplan et al. [8].

Figure 4 contains two useful messages. First, the left panel shows that larger non-embedding models improve not only on WebText2 but also on other text distributions; the curves have different offsets, but they move down together as $N_{\mathrm{K}}$ increases. Second, the right panel shows that lower in-domain WebText2 validation loss corresponds to lower out-of-domain validation loss, both for checkpoints along a single training run and for separately converged models.

The absence of an upward turn should not be read as a statement that training loss can decrease forever without overfitting. The horizontal axis in the right panel is WebText2 validation loss, not raw training-set loss or training time. If a checkpoint truly overfit WebText2 training data, its held-out WebText2 loss would stop improving or increase, and it would move rightward or fail to move left on this plot. Kaplan et al. instead observe that, over their measured regime, improvements in held-out WebText2 loss remain aligned with improvements on the other held-out text distributions.

## 3.3 Joint dependence on model size and data

### 3.3.1 The two edge laws

Each one-dimensional fit hides a second resource that is assumed not to be the bottleneck. The model-size law assumes enough data, while the data-size law assumes enough model capacity:

$$\mathcal{L}(N_{\mathrm{K}}, \infty) = \left(\frac{N_c}{N_{\mathrm{K}}}\right)^{\alpha_N}, \qquad \mathcal{L}(\infty, D) = \left(\frac{D_c}{D}\right)^{\alpha_D}. \tag{35}$$

The infinities are idealized non-bottlenecked limits, not experiments with literally infinite resources. A joint law $\mathcal{L}(N_{\mathrm{K}}, D)$ must recover both edges and describe the bend between them as the active bottleneck changes.

Kaplan et al. use three construction requirements: the constants should absorb an overall rescaling of loss caused by tokenization; the two limits in Eq. (35) must be recovered; and, at fixed finite $N_{\mathrm{K}}$, the correction near $D = \infty$ should have an ordinary integer-power expansion in $1/D$. The data distribution, tokenizer, architecture, and training protocol otherwise remain fixed.

The third requirement is just a smoothness assumption. At fixed $N_{\mathrm{K}}$, set $x = 1/D$. Then "$D$ is very large" becomes "$x$ is close to zero." If the loss is smooth there, its Maclaurin expansion has the form

$$\mathcal{L}(N_{\mathrm{K}}, D) = \mathcal{L}(N_{\mathrm{K}}, \infty) + \frac{a_1(N_{\mathrm{K}})}{D} + \frac{a_2(N_{\mathrm{K}})}{D^2} + \cdots. \tag{36}$$

So this condition says only that, once data is already abundant, the remaining finite-data penalty begins with a term proportional to $1/D$. It does not say that the whole data-scaling curve is $1/D$. A useful statistical intuition is that parameter-estimation error often scales as $1/\sqrt{D}$; near a smooth loss minimum, excess loss is quadratic in that error and therefore scales as $1/D$. This is motivation, not a proof.

### 3.3.2 Constructing the joint law

The goal is to fit one joint function $\mathcal{L}(N_{\mathrm{K}}, D)$ to the full two-dimensional grid. Kaplan et al. choose

$$\boxed{\mathcal{L}(N_{\mathrm{K}}, D) = \left[\left(\frac{N_c}{N_{\mathrm{K}}}\right)^{\alpha_N/\alpha_D} + \frac{D_c}{D}\right]^{\alpha_D}.} \tag{37}$$

We now explain how the two edge laws motivate this form.

Define the transformed loss $R := \mathcal{L}^{1/\alpha_D}$. Taking this power makes the data boundary linear in $1/D$, while transforming the model boundary by the same rule:

$$R(N_{\mathrm{K}}, \infty) = \left(\frac{N_c}{N_{\mathrm{K}}}\right)^{\alpha_N/\alpha_D}, \qquad R(\infty, D) = \frac{D_c}{D}. \tag{38}$$

The simplest joint expression in this transformed coordinate is their sum. Raising it back to the power $\alpha_D$ recovers Eq. (37).

When $D \to \infty$, the data term is small. To inspect this limit, hold $N_{\mathrm{K}}$ fixed and substitute $x = 1/D$ into Eq. (37). Its right-hand side becomes the one-variable function

$$F_{N_{\mathrm{K}}}(x) := \left[\left(\frac{N_c}{N_{\mathrm{K}}}\right)^{\alpha_N/\alpha_D} + D_c x\right]^{\alpha_D}, \qquad \mathcal{L}(N_{\mathrm{K}}, D) = F_{N_{\mathrm{K}}}(1/D). \tag{39}$$

Now $D \to \infty$ is the ordinary expansion point $x \to 0$. The value and slope of $F_{N_\mathrm{K}}$ at zero are

$$\begin{aligned} F_{N_\mathrm{K}}(0) &= \left(\frac{N_c}{N_\mathrm{K}}\right)^{\alpha_N} = \mathcal{L}(N_\mathrm{K}, \infty), \\ F'_{N_\mathrm{K}}(0) &= \alpha_D D_c \left(\frac{N_c}{N_\mathrm{K}}\right)^{\frac{\alpha_N}{\alpha_D}(\alpha_D - 1)}. \end{aligned} \tag{40}$$

Applying the Maclaurin formula $F(x) - F(0) = F'(0)x + O(x^2)$ and substituting $x = 1/D$ back gives

$$\begin{aligned} \mathcal{L}(N_\mathrm{K}, D) - \mathcal{L}(N_\mathrm{K}, \infty) &= F_{N_\mathrm{K}}(1/D) - F_{N_\mathrm{K}}(0) \\ &= \alpha_D \left(\frac{N_c}{N_\mathrm{K}}\right)^{\frac{\alpha_N}{\alpha_D}(\alpha_D - 1)} \frac{D_c}{D} + O(D^{-2}). \end{aligned} \tag{41}$$

Thus the finite-data gap begins at $1/D$.

This also explains why the two edge losses are not added directly. The naive alternative

$$\mathcal{L}_{\text{naive}}(N_\mathrm{K}, D) = \left(\frac{N_c}{N_\mathrm{K}}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D} \tag{42}$$

has a finite-data gap proportional to $D^{-\alpha_D}$ rather than an integer-power expansion in $1/D$. The three requirements do not uniquely derive Eq. (37); they motivate this simplest interpolation, whose real test is the two-dimensional fit.

### 3.3.3 How the fit is run and what the result shows

Kaplan et al. choose a grid of model sizes $N_i$ and dataset sizes $D_j$. For each pair $(N_i, D_j)$, they train a model on the corresponding WebText2 subset, stop at the best held-out loss, and record $\mathcal{L}_{ij} = \mathcal{L}(N_i, D_j)$. The result is a two-dimensional table: each row fixes a dataset size and varies the model, while each column fixes a model size and varies the data. They fit all entries at once with Eq. (37); this is one nonlinear regression, not a separate line fit for every row. The four fitted parameters are

$$\alpha_N = 0.076, \qquad \alpha_D = 0.103, \qquad N_c = 6.4 \times 10^{13}, \qquad D_c = 1.8 \times 10^{13}. \tag{43}$$

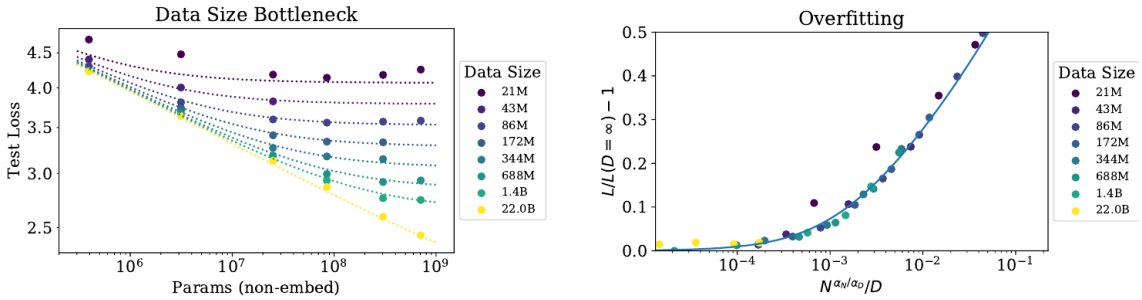



Figure 5: Joint model-data scaling. Left: at fixed finite $D$, increasing model size eventually reaches a data-limited plateau. Right: measurements from different $N_{\mathrm{K}}$ and $D$ collapse when the relative finite-data penalty $\mathcal{L}(N_{\mathrm{K}}, D)/\mathcal{L}(N_{\mathrm{K}}, \infty) - 1$ is plotted against $N_{\mathrm{K}}^{\alpha_N/\alpha_D}/D$. Both horizontal axes are logarithmic; the left loss axis is logarithmic, while the right relative-penalty axis is linear. Reproduced from Figure 9 of Kaplan et al. [8].

**Left panel: which resource is the bottleneck?** Each color holds $D$ fixed and increases $N_{\mathrm{K}}$. For the largest datasets, $D_c/D$ is small and the points stay close to the infinite-data model-size law $\mathcal{L}(N_{\mathrm{K}}, \infty) = (N_c/N_{\mathrm{K}})^{\alpha_N}$. This is the descending lower boundary in the panel. Its straight-line form is consistent with a power law on the panel's log–log axes.

For the smaller datasets, the curves bend above this boundary and flatten earlier. These models are data-limited: after the model becomes large enough, adding parameters brings little improvement because the same finite dataset is being reused. The plateau at the far right approaches $\mathcal{L}(\infty, D) = (D_c/D)^{\alpha_D}$. Looking across the plateau heights for different $D$ therefore reveals the data-size law. The grid needs both large-$D$ runs to expose the model boundary and large-$N_{\mathrm{K}}$ runs to expose the data boundary; the interior points determine how the two regimes join.

**Right panel: does the interior collapse?** The right panel tests whether the joint law really learned the tradeoff between model size and data size. Recall Eq. (37):

$$\mathcal{L}(N_{\mathrm{K}}, D) = \left[ \left( \frac{N_c}{N_{\mathrm{K}}} \right)^{\alpha_N/\alpha_D} + \frac{D_c}{D} \right]^{\alpha_D}. \qquad \text{(37, recalled)}$$

To make the exponents easier to read, define $p := \alpha_N/\alpha_D$.

We first construct the vertical coordinate

$$y := \frac{\mathcal{L}(N_{\mathrm{K}}, D)}{\mathcal{L}(N_{\mathrm{K}}, \infty)} - 1. \qquad (44)$$

Here $\mathcal{L}(N_{\mathrm{K}}, \infty)$ is the loss of the *same-size model* with unlimited data. Dividing by it removes the loss caused by limited model size; the remainder measures only the relative penalty caused by limited data. Thus $y = 0$ means no finite-data penalty, while $y = 0.1$ means that finite data raises the loss by 10%.

Using Eq. (37) and $\mathcal{L}(N_{\mathrm{K}}, \infty) = [(N_c/N_{\mathrm{K}})^p]^{\alpha_D}$, we obtain

$$\begin{aligned} y &= \frac{[(N_c/N_{\mathrm{K}})^p + D_c/D]^{\alpha_D}}{[(N_c/N_{\mathrm{K}})^p]^{\alpha_D}} - 1 \\ &= \left[1 + \frac{D_c}{N_c^p}\frac{N_{\mathrm{K}}^p}{D}\right]^{\alpha_D} - 1. \end{aligned} \tag{45}$$

The last line shows that $N_{\mathrm{K}}$ and $D$ appear only through the ratio $N_{\mathrm{K}}^p/D$. This suggests the horizontal coordinate

$$x := \frac{N_{\mathrm{K}}^p}{D} = \frac{N_{\mathrm{K}}^{\alpha_N/\alpha_D}}{D}, \qquad y = \left(1 + \frac{D_c}{N_c^p}x\right)^{\alpha_D} - 1. \tag{46}$$

Equation (46) is now a one-variable relation $y = f(x)$. Therefore every original experiment $(N_i, D_j, \mathcal{L}_{ij})$ can be converted into one point $(x_{ij}, y_{ij})$. If Eq. (37) is correct, two experiments with the same $x$ must have the same $y$, even when their model and dataset sizes are different. All points should therefore fall on one curve rather than remain as the many separate curves seen in the left panel.

This is the curve collapse shown in the right panel. Small $x$ means abundant data relative to model size, so $y$ is near zero. Large $x$ means too little data for the model, so $y$ rises. The approximate collapse supports the joint law; a failure to collapse would mean that it had modeled the interior tradeoff incorrectly. Since $p \approx 0.74$, doubling model size requires about $2^{0.74} \approx 1.67$ times as much data to keep $x$, and hence the relative finite-data penalty, unchanged.

Equation (46) also tells us where the data-limited regime begins. Choose a maximum acceptable relative penalty $\epsilon$. Kaplan et al. use approximately $\epsilon = 0.02$. With $c := D_c/N_c^p$, Eq. (46) becomes $y = (1 + cN_{\mathrm{K}}^p/D)^{\alpha_D} - 1$. Requiring $y \le \epsilon$ and solving for $D$ gives

$$\begin{gathered} y \le \epsilon \quad \Longleftrightarrow \quad D \ge \frac{c}{(1+\epsilon)^{1/\alpha_D} - 1} N_{\mathrm{K}}^p, \\ p = \frac{0.076}{0.103} \approx 0.738, \qquad \frac{c}{(1.02)^{1/0.103} - 1} \approx 5.5 \times 10^3, \\ \boxed{D \gtrsim (5 \times 10^3) N_{\mathrm{K}}^{0.74} \quad \Longleftrightarrow \quad N_{\mathrm{K}} \lesssim \left(\frac{D}{5 \times 10^3}\right)^{1/0.74}}. \end{gathered} \tag{47}$$

The final line rounds the coefficient and exponent because the fitted parameters themselves are empirical estimates. Thus Eq. (47) is not a new fit: it is Eq. (46) evaluated at the chosen tolerance $y = 0.02$.

**What Eq. (47) gives us.** Here $D$ is the dataset size measured in *tokens after tokenization*, not the number of documents or training sequences. If a dataset contains $M$ sequences of roughly $T$ tokens each, then $D \approx MT$. These are token occurrences, not distinct vocabulary entries: repeated words still contribute one token at every position where they appear. Training for another epoch processes the same tokens again but does not increase $D$; it increases the processed-token count $E$, introduced in the next subsection. The numerical coefficient $5 \times 10^3$ therefore assumes that both $D$ and the non-embedding parameter count $N_{\mathrm{K}}$ are entered as raw counts.

The left form answers: for a model with $N_{\mathrm{K}}$ non-embedding parameters, how many dataset tokens are needed to keep the measured finite-data penalty small? The right form answers the reverse question: given $D$ distinct tokens, approximately how large can the model become before data limitation becomes visible?

This is a data-compatible scale, not a hard upper bound or a compute-optimal model size. A larger model can still be trained, but its loss will bend toward the data-limited plateau, so additional parameters bring diminishing returns. The run-planning procedure in Section 3.5 combines this data check with compute and hardware constraints. Finally, the numerical constant and exponent are historical WebText2 estimates, not a universal modern LLM or DiT prescription; they should be refitted for a new setting.

## 3.4 Critical batch size

Section 3.3 answered an endpoint question: for fixed model size $N_{\mathrm{K}}$ and dataset size $D$, what final loss is attainable? It also tells us whether a proposed target is feasible: when the same loss metric is used, the target must lie above the attainable floor. It does not tell us how expensive that target is to reach.

This subsection asks the next question: after fixing $N_{\mathrm{K}}$, $D$, and one reachable target loss $\ell$, how does the batch size $B$ trade serial updates against total compute? We do not fix compute yet. Instead, we sweep $B$, record the updates $S$ needed to first reach $\ell$, and compute $E = BS$. The result is a speed–compute trade-off: it tells us which batches save steps, which save compute, and where further data parallelism begins to give diminishing returns.

Here $\ell$ is a threshold along the training trajectory, not necessarily the final loss. If loss falls from 10 to 6 and then to 4, the run crosses $\ell = 6$ earlier and $\ell = 4$ later. If we only cared about one constant-batch run taken to its endpoint, measuring the final threshold would be enough. Kaplan et al. instead need $B_{\mathrm{crit}}(\ell)$ along the whole curve because Section 3.5 will compare models stopped at different intermediate losses under the same compute budget.

We use

- $B$ is the number of tokens in one parameter update;
- $S$ is the number of serial parameter updates;
- $E = BS$ is the total number of processed tokens, and $C_{\mathrm{K}} \approx 6N_{\mathrm{K}}E$ is the leading compute.

Here $D$ and $E$ are different: $D$ is the size of the available dataset, whereas $E$ counts every token processed during training, including repeated epochs. Kaplan's joint $N$–$D$ law uses early-stopped test loss, whereas its critical-batch experiment is indexed by training loss; directly connecting their numerical values requires a matched loss metric or an empirical mapping.

**From the observation to an empirical model** The starting observation is simple. With a small batch, the gradient is noisy, so reaching $\ell$ takes many updates. Increasing $B$ reduces this noise and therefore reduces $S$, but the improvement eventually saturates: even a nearly noise-free gradient still needs some irreducible number of updates $S_{\min}$.

There is a reason to try a $1/B$ correction. At one fixed parameter value, suppose the per-example gradients $g_1, \ldots, g_B$ are independently sampled from the same distribution, with $\mathrm{Var}(g_i) = \sigma_g^2$.

The batch gradient is their average, so

$$\begin{aligned} \bar{g}_B &= \frac{1}{B}\sum_{i=1}^{B} g_i, \\ \mathrm{Var}(\bar{g}_B) &= \frac{1}{B^2}\,\mathrm{Var}\left(\sum_{i=1}^{B} g_i\right) \\ &= \frac{1}{B^2}\sum_{i=1}^{B}\mathrm{Var}(g_i) \qquad (g_i \text{ independent}) \\ &= \frac{B\sigma_g^2}{B^2} = \frac{\sigma_g^2}{B}. \end{aligned} \tag{48}$$

This is the variance-of-an-average formula. Independence makes the covariance terms vanish; identical distributions make all $B$ variances equal. For a vector gradient, the same calculation applies to its covariance matrix. Gradients from nearby tokens can be correlated in practice, so the exact $1/B$ law is an approximation and $B$ should be understood as an effective batch size.

The typical noise amplitude is the standard deviation $\sigma_g/\sqrt{B}$, while its variance, $\sigma_g^2/B$, measures the noise power. A larger batch reduces both, making each update a more reliable estimate of the full-data update. It is therefore reasonable to expect fewer noise-induced updates. But this is the extra modeling assumption: Eq. (48) alone does not prove a law for the number of optimization steps. In the local noisy-optimization model, the excess step cost above the noise-free floor is assumed to be approximately proportional to the noise variance. For a fixed target loss and a consistently tuned optimizer, this gives the empirical interpolation [9]

$$\begin{aligned} \underbrace{S(B) - S_{\min}}_{\text{extra steps from noise}} \;\propto\; \underbrace{\mathrm{Var}(\bar{g}_B)}_{\text{noise power}} \;\propto\; \frac{1}{B}, \\ S(B) \approx S_{\min} + \frac{A}{B}, \\ E(B) = BS \approx BS_{\min} + A. \end{aligned} \tag{49}$$

The constant $A$ absorbs the target loss, curvature, learning rate, and other details; it is not numerically equal to $\sigma_g^2$. This construction has two limits. When $B$ is very large, $S(B) \to S_{\min}$. When $B$ is very small, $S(B) \approx A/B$ and hence $E(B) = BS(B) \to A$. We therefore name this small-batch limit

$$E_{\min} := \lim_{B\to 0} E(B) = A. \tag{50}$$

This is an ideal extrapolated limit: an actual batch cannot contain fewer than one training example.

**Define the crossover** Return to Eq. (49). The critical batch is defined as the point at which its irreducible term $S_{\min}$ and noise-related term $A/B$ are equal. Writing that equality first and then solving for the newly defined quantity gives

$$S_{\min} = \frac{A}{B_{\mathrm{crit}}} \quad\Longrightarrow\quad \boxed{B_{\mathrm{crit}}(\ell) := \frac{A}{S_{\min}} = \frac{E_{\min}}{S_{\min}}.} \tag{51}$$

Thus $S_{\min}B_{\mathrm{crit}} = A$ is not an extra assumption. It follows from the definition of the crossover together with $E_{\min} = A$. Since $C_{\mathrm{K}} \approx 6N_{\mathrm{K}}E$, define the minimum-compute limit $C_{\min} := 6N_{\mathrm{K}}E_{\min}$. Substituting these quantities back into Eq. (49) gives

$$\boxed{\begin{aligned} \frac{S}{S_{\min}} &= 1 + \frac{B_{\mathrm{crit}}}{B}, & \frac{E}{E_{\min}} &= \frac{C_{\mathrm{K}}}{C_{\min}} = 1 + \frac{B}{B_{\mathrm{crit}}}, \\ B = B_{\mathrm{crit}} \quad\Longrightarrow\quad S &= 2S_{\min}, & E &= 2E_{\min}, \quad C_{\mathrm{K}} = 2C_{\min}. \end{aligned}} \tag{52}$$

The equality-of-terms criterion is a convenient definition of the crossover, not a theorem that identifies a unique optimal batch. Therefore $B_{\text{crit}}$ achieves neither the fewest updates nor the fewest processed tokens. Choosing $B \gg B_{\text{crit}}$ moves $S$ toward $S_{\min}$ but wastes compute; choosing $B \ll B_{\text{crit}}$ moves $E$ toward $E_{\min}$ but requires many serial updates. The critical batch is the knee at which increasing the batch begins to give sharply diminishing step-count returns.

**What this definition does—and does not—give us** Equation (51) does not predict a numerical batch size from nothing. Before $A$ and $S_{\min}$ have been measured, the ratio $A/S_{\min}$ is also unknown. Its first purpose is to summarize one batch-size sweep by an interpretable number: the location of the knee. For a run aimed at one fixed target loss $\ell$, choosing $B$ near this knee gives a reasonable compromise between serial steps and processed tokens.

The equation $S(B) = S_{\min} + A/B$ contains two unknown constants, $S_{\min}$ and $A$. Choose two different batches $B_1$ and $B_2$, and measure the steps $S_1$ and $S_2$ needed to reach the same loss $\ell$. In an ideal noise-free world, these two runs give two equations, so

$$A = \frac{S_1 - S_2}{1/B_1 - 1/B_2}, \qquad S_{\min} = S_1 - \frac{A}{B_1}. \tag{53}$$

Equivalently, plot $S$ against $1/B$: the intercept is $S_{\min}$ and the slope is $A$. Real training measurements fluctuate and the interpolation is only approximate, so in practice we run several batch sizes and fit a line rather than trusting exactly two points. The fitted ratio $B_{\text{crit}}(\ell) = A/S_{\min}$ is one measured data point. If this full sweep had to be repeated before every large run, however, it would save little work. We therefore need a second step: turn these pilot measurements into a rule that predicts the knee for later runs.

**From pilot measurements to a reusable rule** One training trajectory crosses many loss thresholds, and each threshold has its own knee. Kaplan et al. therefore measure $B_{\text{crit}}$ at many thresholds (the procedure is described with Figure 6 below) and fit the results with a power law of the same form as the earlier laws, writing loss as a function of the resource:

$$\ell(B_{\text{crit}}) = \left(\frac{B_*}{B_{\text{crit}}}\right)^{\alpha_B}, \qquad B_* \approx 2 \times 10^8 \text{ tokens}, \qquad \alpha_B \approx 0.21. \tag{54}$$

In practice, however, the question runs the other way. Before training we choose a target loss $\ell$ and want to know which batch to use. We therefore solve Eq. (54) for $B_{\text{crit}}$: raise both sides to the power $1/\alpha_B$, giving $\ell^{1/\alpha_B} = B_*/B_{\text{crit}}$, so

$$\boxed{B_{\text{crit}}(\ell) = \frac{B_*}{\ell^{1/\alpha_B}} \approx B_* \, \ell^{-4.8}.} \tag{55}$$

In Figure 6, each blue or orange point comes from a batch sweep at a target loss: fit Eq. (53) and read off $B_{\text{crit}} = A/S_{\min}$. The 3M- and 85M-parameter curves nearly overlap, suggesting that loss matters more than model size within this comparison. Both follow the dashed power law of Eq. (55) over losses of roughly 3 to 10.

The green points come from a second, cheaper method that needs no batch sweep. Let $G$ be the true gradient over all data and $\Sigma$ the covariance of the per-example gradients. For each gradient coordinate, the average of the square equals the square of the average plus the variance, and Eq. (48) says the variance of a batch average is divided by $B$. Summing over coordinates gives

$$\mathbb{E}\,\|\bar{g}_B\|^2 = \underbrace{\|G\|^2}_{\text{signal}} + \underbrace{\frac{\operatorname{tr}\Sigma}{B}}_{\text{noise}}, \qquad B_{\text{simple}} := \frac{\operatorname{tr}\Sigma}{\|G\|^2}, \tag{56}$$

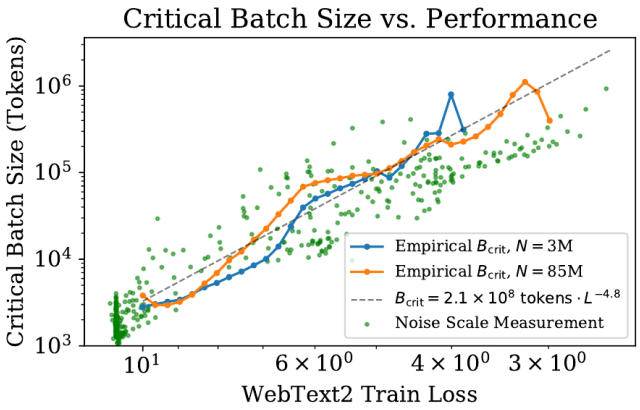



Figure 6: Critical batch size measured for 3M- and 85M-parameter models, together with gradient-noise-scale measurements. The model-size curves approximately share a loss-dependent trend. Reproduced from Figure 10 of Kaplan et al. [8].

where $\operatorname{tr}\Sigma$ is the sum of the variances of all gradient coordinates. The *simplified gradient noise scale* $B_{\text{simple}}$ is the batch at which these noise and signal terms are equal. Below it, noise dominates this measurement; above it, averaging more examples gives a smaller relative improvement in gradient accuracy. This makes it a useful proxy for $B_{\text{crit}}$, not an alternative definition of it: gradient accuracy alone does not determine progress in loss. McCandlish et al.'s fuller noise scale also accounts for the curvature of the loss surface, and even the ratio of $B_{\text{simple}}$ to measured $B_{\text{crit}}$ varies across tasks [9]. Calibrate the proxy against a batch sweep before using it to choose a batch.

The simplified scale can be estimated inside one ordinary training run: measure $\|\bar{g}_B\|^2$ at two batch sizes, for example on one device and on the full data-parallel batch. As in Eq. (53), two measurements give two equations for the two unknowns $\|G\|^2$ and $\operatorname{tr}\Sigma$. Each green point is one such estimate, plotted against the loss reached at that moment.

The green points follow a similar trend without using the batch sweeps, providing an independent check. Kaplan et al. nevertheless use the directly measured $B_{\text{crit}}$ in later analysis. Its power law remains an empirical fit over a limited range and two model sizes, not a consequence of the definition of $B_{\text{crit}}$.

What is Eq. (55) good for? Once fitted, it replaces the batch sweep: given a target loss $\ell$, it predicts the knee directly. For a constant-batch run aimed at final loss $\ell$, it suggests a batch near $B_{\text{crit}}(\ell)$, subject to hardware constraints. This is a time–compute compromise, not the batch that minimizes the number of steps: in the idealized model, taking $B \gg B_{\text{crit}}$ moves closer to the minimum step count. Because the knee grows as loss falls, a batch suited to late training can be unnecessarily large early in training; this motivates increasing the batch during training. The same inverted form is also what Section 3.5 needs: there, model size, effective updates, and batch are all written as functions of the target loss and then multiplied to obtain compute. The numerical fit is not universal and should be remeasured when the data, architecture, or optimizer changes.

**Summary** For a fixed model and target loss, the steps follow $S(B) \approx S_{\min} + A/B$ (Eq. (49)): a larger batch always saves steps but costs more compute. The critical batch $B_{\text{crit}} = A/S_{\min}$ (Eq. (51)) marks the knee where the extra steps caused by noise equal the irreducible steps,

so $S = 2S_{\min}$ and $E = 2E_{\min}$. Above the knee, a larger batch buys few steps; below it, a smaller batch saves little compute. The knee cannot be computed in advance. It is measured by sweeping $B$, recording the steps to each target loss, and fitting $S$ against $1/B$. Repeating this across losses gives $B_{\text{crit}}(\ell) \approx B_* \ell^{-4.8}$ (Eq. (55)), which depends mainly on the loss reached and hardly on model size. This law has two uses: it picks a batch near the knee without a new sweep, growing as the loss falls, and it lets Section 3.5 convert any run into the batch-independent coordinates $S_{\min}$ and $C_{\min}$.

### 3.5 Fixed compute: how large should the model be?

This subsection answers one question: with a fixed training compute, how should we choose the model size $N_{\text{K}}$, the batch $B$, and the number of updates $S$? Data is assumed plentiful, so the choice is between model size and training length, not dataset size; the end of this subsection checks this assumption.

The ideal experiment is simple. Train models of many sizes. For a budget $C_{\text{K}}$, the relation $C_{\text{K}} = 6N_{\text{K}}BS$ fixes $BS$ for each size, and with $B$ fixed it fixes $S$. Compare the losses the models have reached at those steps and keep the lowest. Kaplan et al. have exactly such data: models of many sizes, nearly all trained with $B = 2^{19}$ tokens. Why not compare them directly?

Because the comparison would be unfair. Reaching a loss $\ell$ takes $S = S_{\min} + A/B$ steps (Eq. (49)), so

$$C_{\text{K}} = 6N_{\text{K}}BS = 6N_{\text{K}}\left(BS_{\min} + A\right). \tag{57}$$

For the same model and the same loss, a larger batch costs more compute and a smaller one more steps. A fixed $B = 2^{19}$ sits at a different point of this trade-off for each checkpoint: early in training, when the loss is high and $B_{\text{crit}}$ is small, it is too large; late in training it is too small. Models of different sizes reach a given loss at different times, so raw compute penalizes them by different amounts. The fair setting puts every checkpoint at its own knee, $B = B_{\text{crit}}(\ell)$, so that all of them sit at the same point of their own trade-off. Section 3.4 makes this practical: $B_{\text{crit}}$ depends on the loss but hardly on model size, so one formula gives it for every model.

**Convert every checkpoint to the critical batch** Recall Eqs. (52) and (55). Since $B_{\text{crit}} = A/S_{\min}$, the step law becomes $S = S_{\min} + S_{\min}B_{\text{crit}}/B$, which is the first ratio below; dividing Eq. (57) by $C_{\min} = 6N_{\text{K}}A$ gives the second:

$$\frac{S}{S_{\min}} = 1 + \frac{B_{\text{crit}}}{B}, \qquad \frac{C_{\text{K}}}{C_{\min}} = 1 + \frac{B}{B_{\text{crit}}}, \tag{52}$$

$$B_{\text{crit}}(\ell) = \frac{B_*}{\ell^{1/\alpha_B}} \approx B_*\, \ell^{-4.8}. \tag{55}$$

Take one checkpoint $(\ell, N_{\text{K}}, B, S, C_{\text{K}})$ of an existing run. Equation (55) gives $B_{\text{crit}}(\ell)$ when $\ell$ uses the loss convention on which that law was fitted. Plentiful data can make training and test losses close, but evaluation-mode differences must still be matched or calibrated before substituting one for the other. Each relation in Eq. (52) then has only one unknown:

$$\boxed{S_{\min} = \frac{S}{1 + B_{\text{crit}}(\ell)/B}, \qquad C_{\min} = \frac{C_{\text{K}}}{1 + B/B_{\text{crit}}(\ell)},} \qquad C_{\text{K}} = 6N_{\text{K}}BS. \tag{58}$$

Setting $B = B_{\text{crit}}$ in Eq. (52) gives $S = 2S_{\min}$ and $C_{\text{K}} = 2C_{\min}$: trained at its knee, this checkpoint would have needed $2S_{\min}$ steps and $2C_{\min}$ compute. Kaplan et al. record $S_{\min}$ and $C_{\min}$ instead. These differ by the same factor of two for every checkpoint, so they rank models identically; a real budget $C_{\text{K}}$ spent at the knee simply corresponds to $C_{\min} = C_{\text{K}}/2$.

**Compare model sizes at the same compute** Every checkpoint now carries its loss $\ell$, model size $N_{\mathrm{K}}$, $S_{\min}$, and $C_{\min}$, so the ideal experiment can be carried out. Fix a budget $C_{\min}$. On each model's converted curve, find the checkpoint at which $C_{\min}$ reaches this budget, and read its loss $\ell$ and model size $N_{\mathrm{K}}$. A small model has taken many steps but stalls at a high floor; a large model has a lower floor but has taken few steps. Plotted against $N_{\mathrm{K}}$, these losses therefore form a U.

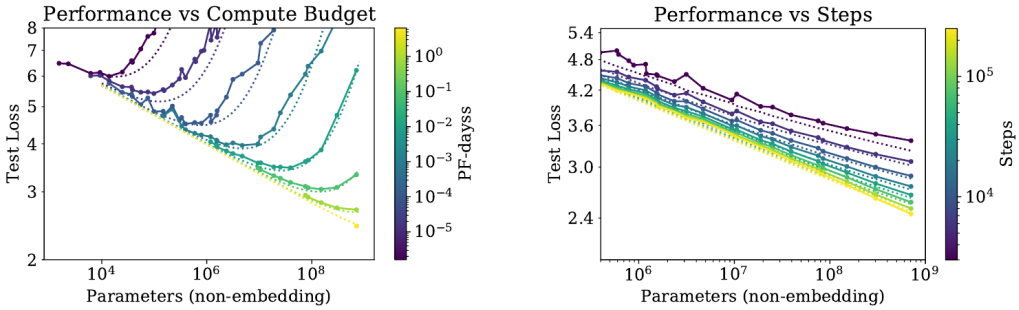



Figure 7: Loss versus non-embedding model size at fixed compute budgets (left) and fixed numbers of training steps (right). Solid curves are experiments and dotted curves are predictions from the learning-curve model. Reproduced from Figure 11 of Kaplan et al. [8].

The left panel of Figure 7 shows these U-shaped curves, one color per budget. It is plotted against Kaplan's raw compute; converting to $C_{\min}$ moves the points by different factors but keeps the U. The right panel fixes the number of steps instead and has no minimum, so the U comes from the compute constraint.

The bottom of each U gives two numbers for its budget: a loss $\ell^*$ and a model size $N_{\mathrm{K}}^*$. This $\ell^*$ is not the converged loss of $N_{\mathrm{K}}^*$; it is the lowest loss any model can reach with this budget, and $N_{\mathrm{K}}^*$ is the size that reaches it. Repeating this for many budgets and plotting the bottoms against $C_{\min}$ gives two straight lines on log-log axes (Figure 8), that is, two power laws. To insert their numerical coefficients without mixing units, define the dimensionless PF-day coordinate

$$c_{\min} := \frac{C_{\min}}{8.64 \times 10^{19}\ \mathrm{FLOPs}}. \tag{59}$$

The historical fits are

$$\boxed{N_{\mathrm{K}}^* \approx 1.3 \times 10^9\, c_{\min}^{0.73}, \qquad \ell^* \approx \left(\frac{3.1 \times 10^8}{c_{\min}}\right)^{0.050}}. \tag{60}$$

The second is the batch-corrected frontier of Eq. (34).

**Choose the batch and the steps** With $N_{\mathrm{K}}^*$ and $\ell^*$ known, $B$ and $S$ follow by substitution. Choose the constant batch to equal the critical batch for the predicted *endpoint* loss: insert $\ell^*$ from Eq. (60) into Eq. (55). This is a time–compute compromise; $C_{\mathrm{K}}$ alone does not uniquely determine $N_{\mathrm{K}}$, $B$, and $S$. Under the endpoint-knee convention, $C_{\mathrm{K}} = 2C_{\min}$, so a given real budget gives $c_{\min} = C_{\mathrm{K}}/(2 \times 8.64 \times 10^{19}\ \mathrm{FLOPs})$. The batch in tokens and the updates then

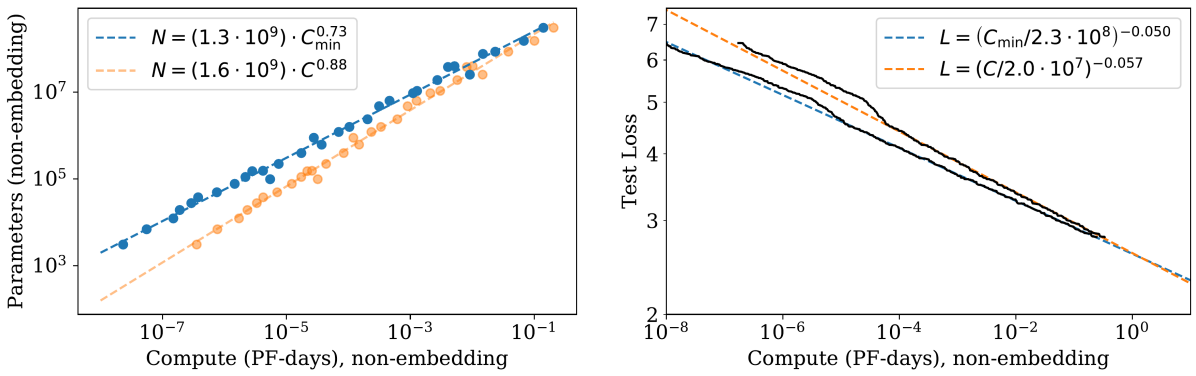



Figure 8: The bottoms of the fixed-budget U-curves versus compute budget. Left: the optimal model size $N_{\mathrm{K}}^*$; blue uses $C_{\min}$, orange raw compute at $B = 2^{19}$. Right: the lowest loss $\ell^*$; the lower curve uses $C_{\min}$, the upper curve raw compute. Its legend scale $2.3 \times 10^8$ differs from the summary value in Eq. (34). Reproduced from Figures 14 (left) and 13 of Kaplan et al. [8].

follow:

$$\boxed{\begin{aligned} B &= B_{\mathrm{crit}}(\ell^*) = B_* \left(\frac{c_{\min}}{3.1 \times 10^8}\right)^{0.050/\alpha_B} \approx 2 \times 10^6 \, c_{\min}^{0.24}, \\ S &= \frac{C_{\mathrm{K}}}{6 N_{\mathrm{K}}^* B} = \frac{2(8.64 \times 10^{19}) c_{\min}}{6 N_{\mathrm{K}}^* B} \approx 1.1 \times 10^4 \, c_{\min}^{0.03}. \end{aligned}} \tag{61}$$

The batch exponent is $4.8 \times 0.050 = 0.24$; the step exponent is what is left, $1 - 0.73 - 0.24 = 0.03$. Multiplying compute by ten multiplies the model size by $10^{0.73} \approx 5.4$, the batch by $10^{0.24} \approx 1.7$, and the steps by only $10^{0.03} \approx 1.07$. Almost all new compute goes into a larger model and a larger batch, not into a longer run.

Two further observations stand out. First, the best model is stopped well before it converges: at $C_{\min} = 1$ PF-day, $N_{\mathrm{K}}^* \approx 1.3 \times 10^9$ stops at $\ell^* \approx 2.66$, whereas Eq. (31) puts the converged loss of that size at about 2.33. Second, compute buys loss slowly: ten times the budget lowers $\ell^*$ by only about 11%. The measured laws record both facts but do not explain them.

After rounding the model to realizable widths and depths and the batch to a hardware-feasible value, recompute $S = C_{\mathrm{K}}/(6 N_{\mathrm{K}} B)$ and the processed-token budget $E = C_{\mathrm{K}}/(6 N_{\mathrm{K}})$. If the chosen batch is no longer near the endpoint knee, $C_{\min} = C_{\mathrm{K}}/2$ is no longer the appropriate conversion. Instead, evaluate feasible model and batch candidates with the fitted learning curve and the actual budget. Check the available dataset separately: repeated passes increase $E$, not Kaplan's $D$.

**Fitting a learning curve for a specific run: $\ell(N_{\mathrm{K}}, S)$** The basic question about a single run is: what loss does a model of size $N_{\mathrm{K}}$ reach after $S$ steps? One formula $\ell(N_{\mathrm{K}}, S)$ cannot answer it for all runs, because $S$ depends on the batch: $S = S_{\min} + A/B$ (Eq. (49)), so the same model at the same loss has taken different numbers of steps in runs with different batches. To compare runs fairly, Kaplan et al. fit $\ell(N_{\mathrm{K}}, S_{\min})$ instead. $S_{\min}$ is the number of steps the model needs to reach a loss with a very large batch, or equivalently half the steps it needs at $B_{\mathrm{crit}}$; it depends only on the model and the loss. Eq. (58) converts every checkpoint of every run to $S_{\min}$, whatever its batch. To predict a specific run with batch $B$, go back: the formula gives the loss $\ell$ after $S_{\min}$ steps, and that run reaches $\ell$ after $S = S_{\min}\,(1 + B_{\mathrm{crit}}(\ell)/B)$ real steps (Eq. (52)).

Two facts fix its form. First, the end point is already known. Trained for ever, a model converges, and the converged loss of each model size is what Eq. (31) measured. So as $S_{\min} \to \infty$ the formula must reduce to $(N_c/N_{\mathrm{K}})^{\alpha_N}$, the model's capacity floor. Second, the gap between a

model's loss and this floor shrinks as a power of $S_{\min}$: plotted on log-log axes, the gap against $S_{\min}$ is a straight line. Floor plus gap gives

$$\boxed{\ell(N_{\mathrm{K}}, S_{\min}) = \underbrace{\left(\frac{N_c}{N_{\mathrm{K}}}\right)^{\alpha_N}}_{\text{capacity floor}} + \underbrace{\left(\frac{S_c}{S_{\min}}\right)^{\alpha_S}}_{\text{optimization error}}}. \tag{62}$$

Fitting its four constants to all converted checkpoints gives

$$\alpha_N = 0.077, \qquad \alpha_S = 0.76, \qquad N_c = 6.5 \times 10^{13}, \qquad S_c = 2.1 \times 10^3. \tag{63}$$

The capacity term nearly coincides with Eq. (31), as the first fact requires.

**Predict a constant-batch trajectory** The frontier $\ell^*(C_{\min})$ is a lower envelope over different model sizes, not the trajectory of one fixed model. For a selected $N_{\mathrm{K}}$ and constant global batch $B$, define $f_N = (N_c/N_{\mathrm{K}})^{\alpha_N}$. A convenient way to draw the reference curve is to choose losses $\ell > f_N$ and compute their predicted crossing steps:

$$\begin{aligned} u(\ell) &= \frac{S_c}{(\ell - f_N)^{1/\alpha_S}}, \\ s(\ell) &= u(\ell)\left(1 + \frac{B_* \ell^{-1/\alpha_B}}{B}\right). \end{aligned} \tag{64}$$

Plot $(s(\ell), \ell)$, or invert this monotone relation numerically to obtain $\ell(s)$. Do not replace $S_{\min}$ by $S/2$ throughout a constant-batch run: that equality holds only at a target whose critical batch equals $B$. A changing batch schedule requires a separately calibrated trajectory model. Exclude warmup and the initial transient, and compare the prediction with the same loss definition, evaluation mode, and data distribution used for the fit. Raw training loss and held-out loss need not coincide, even before overfitting, because dropout and other evaluation settings can differ.

The frontier and trajectory are independently fitted approximations, not exact identities. For example, a real budget of 2 PF-days gives $c_{\min} = 1$, $N_{\mathrm{K}} = 1.3$B, $\ell^* \approx 2.658$, $B \approx 1.902$M tokens, and $S \approx 11{,}646$ updates using the historical frontier and $1/\alpha_B = 1/0.21$. Combining the learning-curve coefficients in Eq. (63) with Eq. (64) predicts an endpoint near 2.738, rather than exactly 2.658. A new study should check this consistency, refit on its own pilots, and quantify prediction error using repeated runs and a held-out larger scale. Randomly withholding checkpoints from the same run is not an independent test of extrapolation across model sizes.

The formula also explains the two observations above, because every point on a U is a point on some model's learning curve. At a fixed budget, $E_{\min} = B_{\mathrm{crit}} S_{\min}$ (Eq. (51)) factors the budget as

$$C_{\min} = 6 N_{\mathrm{K}} B_{\mathrm{crit}} S_{\min}, \tag{65}$$

so a model of size $N_{\mathrm{K}}$ gets $S_{\min} = C_{\min}/(6 N_{\mathrm{K}} B_{\mathrm{crit}})$. Substituting leaves a function of model size alone,

$$\ell(N_{\mathrm{K}} \mid C_{\min}) = \underbrace{\left(\frac{N_c}{N_{\mathrm{K}}}\right)^{\alpha_N}}_{\text{falls as } N_{\mathrm{K}} \text{ grows}} + \underbrace{\left(\frac{6 B_{\mathrm{crit}} S_c\, N_{\mathrm{K}}}{C_{\min}}\right)^{\alpha_S}}_{\text{rises as } N_{\mathrm{K}} \text{ grows}}. \tag{66}$$

Here $B_{\mathrm{crit}}$ is evaluated at the loss reached; near the best model this loss barely changes with $N_{\mathrm{K}}$, so we treat it as a constant. One term falls and the other rises, which is exactly the U of Figure 7. The dotted curves there are this prediction, converted to raw compute with Eq. (58). They match the experiments except at the smallest budgets.

Where is the bottom of the U? Enlarge the model by 1%. At fixed $C_{\min}$, $S_{\min}$ falls by 1%. Since $(1.01)^p \approx 1 + 0.01p$, the capacity term falls by the fraction $0.01\alpha_N$ of itself, while the optimization term rises by the fraction $0.01\alpha_S$ of itself. At the bottom the two changes cancel:

$$\alpha_N \left(\frac{N_c}{N_{\mathrm{K}}^*}\right)^{\alpha_N} = \alpha_S \left(\frac{S_c}{S_{\min}^*}\right)^{\alpha_S} \quad \Longrightarrow \quad \frac{\text{optimization error}}{\text{capacity floor}} = \frac{\alpha_N}{\alpha_S} \approx \frac{0.077}{0.76} \approx 0.10. \tag{67}$$

Setting the derivative of Eq. (66) to zero with Eq. (23) gives the same condition. The compute-optimal model is therefore stopped while its loss is still about 10% above its own converged loss. Training it further would lower the optimization error, but the same compute lowers the loss more when spent on a larger model.

Why is the frontier exponent so small? Equation (67) says that at the optimum both terms of Eq. (62) are fixed fractions of the loss $\ell$. Setting each term proportional to $\ell$, and recalling Eq. (55), every resource grows as a power of $1/\ell$:

$$N_{\mathrm{K}}^* \propto \ell^{-1/\alpha_N}, \qquad S_{\min}^* \propto \ell^{-1/\alpha_S}, \qquad B_{\mathrm{crit}} \propto \ell^{-1/\alpha_B}. \tag{68}$$

Since $C_{\min}$ is their product (Eq. (65)), the inverse exponents add:

$$\begin{aligned} &C_{\min} \propto \ell^{-(1/\alpha_N + 1/\alpha_S + 1/\alpha_B)} \quad \Longrightarrow \quad \ell^* \propto C_{\min}^{-\alpha_C^{\min}}, \\ &\alpha_C^{\min} = \frac{1}{1/\alpha_N + 1/\alpha_S + 1/\alpha_B} \approx \frac{1}{13.0 + 1.3 + 4.8} \approx 0.052. \end{aligned} \tag{69}$$

Lowering the loss requires enlarging all three resources at once, so compute buys loss slowly. The prediction 0.052 is close to the measured 0.050. The formula thus reproduces what was read off the checkpoints and explains Kaplan's headline: train large models and stop them well before convergence.

**Where the rule stops** The rule assumed plentiful data, but Eq. (47) says that keeping the finite-data penalty small requires $D \gtrsim (5 \times 10^3) N_{\mathrm{K}}^{0.74}$. Along the frontier $N_{\mathrm{K}}^* \propto C_{\min}^{0.73}$, so

$$D_{\text{needed}} \propto \left(C_{\min}^{0.73}\right)^{0.74} \approx C_{\min}^{0.54}. \tag{70}$$

whereas the tokens a compute-optimal run processes grow only as $E_{\min}^* = B_{\mathrm{crit}}^* S_{\min}^* \propto C_{\min}^{0.27}$, the sum of the batch and step exponents; training at $B_{\mathrm{crit}}$ doubles $E$ but does not change this exponent. Taken literally at large enough scale, the model would need more distinct data than it ever reads. The measured laws therefore cannot all extrapolate forever; a regime, schedule, or data assumption must eventually change.

## 3.6 What Kaplan has established

Kaplan's results reduce to seven points in three groups: what the loss depends on, how the batch enters training, and how to spend a fixed budget.

### What the loss depends on

1. **Count non-embedding parameters; shape matters little.** At fixed $N_{\mathrm{K}}$, changing depth, width, FFN ratio, or head size over a wide range changes the loss by only a few percent (Figure 1). Indexed by $N_{\text{total}}$, models of different depths fall on different curves; indexed by $N_{\mathrm{K}}$, they collapse onto one (Figure 2). The first claim has largely held. The second has been questioned: Chinchilla counts all parameters, and later work traces the gap between Kaplan's and Chinchilla's allocations to excluding embeddings at small scale [11], or to the last layer's compute, warmup length, and optimizer tuning [12].

2. **Each resource gives a power law, but for a different loss.** When the other resources are not bottlenecks, the loss is a power law in model size, data, and compute. The three losses differ. $\mathcal{L}(N_{\mathrm{K}})$ (Eq. (31)) is the near-converged loss on plentiful data. $\mathcal{L}(D)$ (Eq. (30)) is the best early-stopped loss of a large model on $D$ unique tokens. $\mathcal{L}(C_{\min})$ (Eq. (34)) is not a converged loss but the lowest loss any model reaches with that compute. The joint law (Eq. (37)) combines the first two and gives the data a model needs to avoid overfitting, $D \gtrsim 5 \times 10^3 \, N_{\mathrm{K}}^{0.74}$ (Eq. (47)).

**How the batch enters training**

3. **The batch trades steps against tokens.** To reach a loss $\ell$, a run with batch $B$ needs $S = S_{\min} + A/B$ steps and $E = BS = BS_{\min} + A$ tokens (Eq. (49)). $S_{\min}$ is the fewest steps, reached with a very large batch; $A = E_{\min}$ is the fewest tokens, reached with a very small batch; both depend on $\ell$. A batch above $B_{\mathrm{crit}} = A/S_{\min}$ needs fewer than $2S_{\min}$ steps but more than $2E_{\min}$ tokens; a batch below it, the reverse.

4. $B_{\mathrm{crit}}$ **is the knee, and it makes runs comparable.** At $B_{\mathrm{crit}}$, the extra steps caused by gradient noise, $A/B$, equal the irreducible steps $S_{\min}$ (Eq. (51)). It depends mainly on the loss, hardly on model size, and grows as the loss falls: $B_{\mathrm{crit}} \approx B_* \ell^{-4.8}$ (Eq. (55)). Since $S/S_{\min} = 1 + B_{\mathrm{crit}}/B$ and $C/C_{\min} = 1 + B/B_{\mathrm{crit}}$ (Eq. (52)), any checkpoint can be converted to the batch-independent $S_{\min}$ and $C_{\min}$ (Eq. (58)), so runs with different batches can be compared fairly.

**How to spend a fixed budget**

5. **The recipe.** Converted checkpoints at equal $C_{\min}$ form U-curves, and their bottoms give $N_{\mathrm{K}}^*$ and $\ell^*$ (Eq. (60)). $C_{\min}$ is a bookkeeping value, not compute one can actually spend; a real budget $C_{\mathrm{K}}$ spent at the knee corresponds to $C_{\min} = C_{\mathrm{K}}/2$. Then the model size is $N_{\mathrm{K}}^* \approx 1.3 \times 10^9 \, c_{\min}^{0.73}$, the batch $B = B_{\mathrm{crit}}(\ell^*) \approx 2 \times 10^6 \, c_{\min}^{0.24}$, and the steps $S = C_{\mathrm{K}}/(6 N_{\mathrm{K}}^* B) \approx 1.1 \times 10^4 \, c_{\min}^{0.03}$ (Eq. (61)). The expected learning curve follows from $\ell(N_{\mathrm{K}}, S_{\min})$ (Eq. (62)) and $S = S_{\min}(1 + B_{\mathrm{crit}}(\ell)/B)$, evaluated separately at each loss threshold. Here $c_{\min}$ is defined in Eq. (59); the independently fitted frontier and trajectory need not predict exactly the same endpoint.

6. **Why the recipe looks this way.** Eq. (62) reproduces the U-curves. At the bottom, the optimization error is $\alpha_N/\alpha_S \approx 10\%$ of the capacity floor (Eq. (67)): the best model stops about 10% above its converged loss, hence "train large models and stop early". Lowering the loss requires growing model size, steps, and batch together, so compute buys loss slowly: the predicted exponent $1/(1/\alpha_N + 1/\alpha_S + 1/\alpha_B) \approx 0.052$ (Eq. (69)) is close to the measured 0.050.

7. **Where it stops.** Avoiding overfitting needs $D \propto C_{\min}^{0.54}$ unique tokens (Eq. (70)), but a compute-optimal run processes only $\propto C_{\min}^{0.27}$ tokens. At large enough scale the two contradict, so these laws cannot be extrapolated indefinitely.

These are the results of a method, not universal constants. The constants depend on WebText2, the tokenizer, the 2020 Transformer family, the optimizer and its schedule, and the choice to count non-embedding parameters. The allocation exponent 0.73 in particular did not survive: the more direct experiment of the next section finds $N^* \propto C^{0.49}$. What transfers is the sequence: define the counts, sweep, convert to a common batch, fit the frontier, and test one larger point.

# 4 Chinchilla: Measuring the Allocation Directly

Kaplan's recipe grows the model much faster than the data, $N_{\mathrm{K}}^{*} \propto C_{\mathrm{min}}^{0.73}$, and many large models of 2020–2022 followed it: ever larger, trained on roughly 300B tokens. Hoffmann et al. [6] ask Kaplan's question again—at fixed compute, how large a model and how many tokens?—but measure the answer more directly. They find that parameters and tokens should grow at nearly the same rate. At the compute of Gopher, a 280B model trained on 300B tokens, this calls for a model about four times smaller trained on about four times more tokens. They trained that model, Chinchilla, with 70B parameters and 1.4T tokens, and it beat Gopher in loss and on downstream tasks.

## 4.1 What changes from Section 3.5

Section 3.5 answered the fixed-compute question with runs Kaplan et al. already had: models of many sizes, all trained on the full dataset with one cosine schedule of $2.5 \times 10^5$ steps, about 130B tokens. It converted every checkpoint's compute to $C_{\mathrm{min}}$ with the critical batch (Eq. (58)), fixed a budget $C_{\mathrm{min}}$, read on each model's curve the checkpoint that reaches it, and plotted these losses against $N$ (Figure 7, left); the bottom of each U gave $N^{*}$. Chinchilla asks the same question but trains a separate run for every point: at budget $C$, a model of size $N$ trains on exactly the $C/(6N)$ tokens the budget allows, with a cosine schedule that ends there, and only its final loss is used. Kaplan's points are checkpoints partway through a schedule, before its learning rate has finished decaying; Chinchilla's are finished runs. Hoffmann et al. suggest that this is a major source of the different answer. Porian et al. find otherwise: matching each run's decay to its token budget barely moves the allocation, and once the last layer's compute, warmup length, and optimizer tuning are corrected, even a constant learning rate recovers Chinchilla's [12].

Chinchilla also counts more simply. For planning, its compute is commonly approximated as

$$C \approx 6ND, \qquad D = BS. \tag{71}$$

Here $N$ includes embeddings, and $D$ is the number of tokens processed: the batch in tokens times the number of steps. It is Kaplan's $E$, not his dataset size $D$. Nor is $C$ converted to $C_{\mathrm{min}}$: it is the compute actually spent, and the study never measures $B_{\mathrm{crit}}$. The runs of the scaling study see less than one epoch of data, so the study treats the available data as non-limiting. This does not assert that the corpus contains no duplicated content. The paper's actual FLOP accounting also includes attention-pair, softmax, embedding, and output-projection terms; its Appendix F compares this fuller estimate with $6ND$. A new model family should check the approximation, especially at small size or long context.

The batch is not a variable of the study. Hoffmann et al. set it, like the learning rate, by existing heuristics; the final 70B model uses 1.5M tokens, doubled to 3M midway. Spent compute matches $C_{\mathrm{min}}$ only when the batch is well below $B_{\mathrm{crit}}$, where $C/C_{\mathrm{min}} = 1 + B/B_{\mathrm{crit}} \approx 1$ (Eq. (52)), provided the optimizer is tuned comparably. Chinchilla does not establish that its batches satisfy this condition over the whole trajectory, nor does it derive an optimal batch from $C$ (Section 5.1). The mid-run doubling is compatible with a growing critical batch, but is not a measurement of that relationship. Later work shows that batch changes also require attention to learning rate and Adam's second-moment smoothing parameter $\beta_2$ [12].

## 4.2 Three ways to measure the allocation

Chinchilla measures the allocation directly with IsoFLOP curves, then checks it in two other ways, since one estimate could be an artifact of its fitting procedure.

**IsoFLOP curves** Fix $C$. Choosing a model size then fixes its tokens:

$$D = \frac{C}{6N}. \tag{72}$$

For each budget, train models of several sizes, each for exactly its $D$ tokens, and compare their final losses. A small model sees many tokens but lacks capacity; a large model has capacity but sees few tokens. As in Figure 7 (left), the losses form a U, here called an *IsoFLOP curve* (Figure 9, left), but every point is now a finished run.

Chinchilla runs nine budgets, from $6\times10^{18}$ to $3\times10^{21}$ FLOPs, with models up to 16B parameters. A parabola fitted to each IsoFLOP curve locates its bottom $(N^*, D^*)$. Plotted against $C$ on log-log axes, the bottoms lie on straight lines (Figure 9, center and right):

$$N^* \propto C^{0.49}, \qquad D^* \propto C^{0.51}. \tag{73}$$

Kaplan's split is $N_{\mathrm{K}}^* \propto C^{0.73}$ and $E \propto C^{0.27}$. Chinchilla says to grow parameters and tokens at almost the same rate.

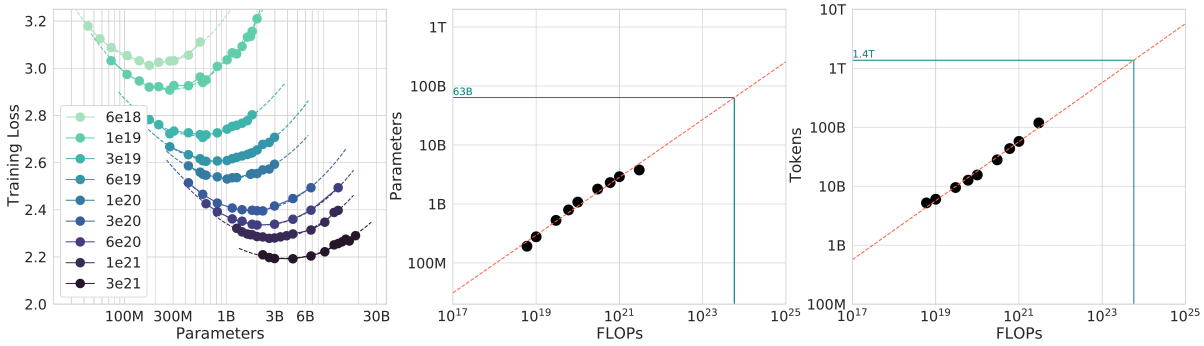



Figure 9: IsoFLOP curves. Left: final loss versus model size at nine fixed budgets; dashed parabolas locate each minimum. Center and right: the minima against compute, with power-law fits extrapolated to Gopher's budget. Reproduced from Figure 3 of Hoffmann et al. [6].

**Lower envelope of learning curves** Train each model size, from 70M to 10B, with four schedule lengths, and at every FLOP count keep the lowest loss over all runs. This is Kaplan's lower envelope, but built from runs whose schedules end near the points used. It gives exponents $(0.50, 0.50)$.

**A fitted loss surface** Each run contributes one point: its size $N$, its tokens $D$, and its final loss. Fit all these points with

$$\widehat{\mathcal{L}}(N, D) = \mathcal{L}_\infty + \frac{A_N}{N^\alpha} + \frac{A_D}{D^\beta} : \tag{74}$$

an irreducible loss, a term that falls with model size, and a term that falls with tokens. Only the last point of a run qualifies: an earlier checkpoint has seen fewer tokens but has not had the decay of a schedule ending there, so its loss lies above the surface. At fixed compute, $D = C/(6N)$ turns the last term into $A_D(6N/C)^\beta$, which rises with $N$, so every budget again has a U. At its bottom, a 1% larger model lowers the model term by the fraction $0.01\alpha$ of itself and raises the data term by the fraction $0.01\beta$ of itself, as in Eq. (67). Setting $\alpha A_N/N^\alpha = \beta A_D(6N/C)^\beta$ and solving for $N$ gives

$$N^*(C) \propto C^{\beta/(\alpha+\beta)}, \qquad D^*(C) \propto C^{\alpha/(\alpha+\beta)}. \tag{75}$$

The fitted $\alpha \approx 0.34$ and $\beta \approx 0.28$ give exponents of about 0.46 and 0.54 using the unrounded fit. All three methods agree: split additional compute roughly equally between parameters and tokens. The claim is "near one half in this regime," not "exactly one half everywhere."

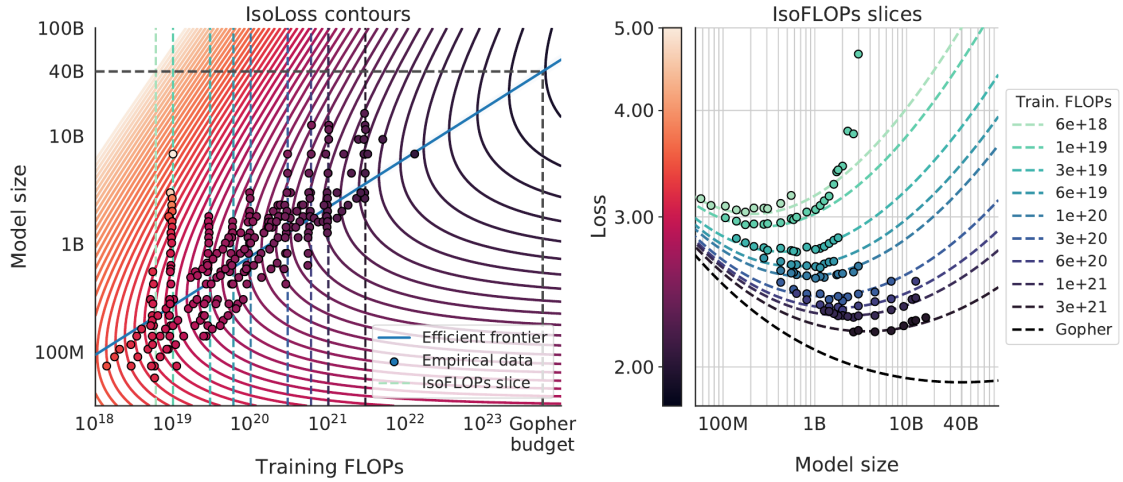



Figure 10: The fitted Chinchilla loss surface (left) and its IsoFLOP slices (right). The blue efficient frontier follows the minimum of each loss contour, while the U-shaped slices expose an interior optimum at fixed compute. Reproduced from Figure 4 of Hoffmann et al. [6].

The proportionality constant is also needed for planning. Under $C = 6ND$, the same stationarity condition gives

$$N^*(C) = \left(\frac{\alpha A_N}{\beta A_D}\right)^{1/(\alpha+\beta)} \left(\frac{C}{6}\right)^{\beta/(\alpha+\beta)}, \qquad D^*(C) = \frac{C}{6N^*(C)}. \tag{76}$$

Choose a hardware-feasible, empirically validated global batch separately (Section 5.1), then set $S = D^*/B$ and make the schedule appropriate to that horizon (Part II, Section 5).

# 5 From Scaling Laws to Scaling Experiments

First choose what batch efficiency means (Section 5.1), then estimate it at the intended training horizon (Section 5.2). Section 5.3 turns these measurements into a practical ladder: allocate the pilot budget, fit the quantities needed for the target, and test the extrapolation before committing the remaining compute.

## 5.1 $B_{\text{opt}}$ and $B_{\text{crit}}$: practical choices

**Start near $B_{\text{opt}}$ for the best loss at fixed compute; try larger batches when shorter training time matters. Use a measured $B_{\text{crit}}$ as a reference upper end of that search.** Beyond the classical knee, additional batch buys progressively smaller step savings per extra token. This is an efficiency guideline, not a stability limit. The definitions below explain when this rule is useful.

**Lowest loss at a fixed budget.** For fixed model size $N$ and processed tokens $D$, define

$$B_{\text{opt}}(N, D) \in \arg\min_{B} \min_{h \in \mathcal{H}} \mathcal{L}(N, D; B, h), \tag{77}$$

where $h$ denotes the hyperparameters allowed to vary in a declared search space $\mathcal{H}$. Compare completed runs with the same schedule family and a comparable tuning budget. Retune LR, and

check decay and optimizer moment timescales: a sweep with all other numbers fixed measures a *conditional* optimum, which may mostly reflect a poorly transferred recipe [12, 1].

**The elbow at a fixed target loss.** The classical $B_{\text{crit}}(\ell)$ instead fixes the model and loss, then fits $S(B) = S_{\min} + E_{\min}/B$. At its knee, $B_{\text{crit}} = E_{\min}/S_{\min}$, both steps and tokens are twice their limiting minima. This is a useful normalization of the trade-off; it is neither a hard stability limit nor necessarily the fastest batch. The idealized law has no small-batch loss penalty. The finite optimum in Eq. (77) therefore needs empirical measurement, not a deduction from that law.

| Quantity | What stays fixed? | What it tells us |
|---|---|---|
| $B_{\text{opt}}$ | Model, tokens, tuning protocol | Best final loss |
| Classical $B_{\text{crit}}$ | Model and target loss | Step–token trade-off knee |
| $B_{\text{ELDE}}$ | Model, tokens, efficiency tolerance | Largest batch passing a relative data-efficiency test |
| Local batch tolerance | Checkpoint, continuation length, evaluation rule | Acceptable batch for that training segment |

Local continuation sweeps can be cheaper than restarting complete runs. Merrill et al. branch from checkpoints and compare larger batches over matched additional tokens [10]. Such a result can motivate a batch schedule, but is specific to the checkpoint and continuation protocol. Clone optimizer state as well as weights, declare how LR and moments are changed, and validate the resulting full schedule. Neither a local tolerance nor the gradient-noise proxy of Section 3.4 is automatically the global knee.

**Use published laws to locate the sweep.** Kaplan's Eq. (55) gives $B_{\text{crit}}$ as a power law in the loss, so the loss must be predicted before the batch can be set. Later studies fit against tokens instead. Varying $N$ and $D$ separately, Zhang et al. find that their measured CBS grows mainly with $D$ and only weakly with $N$ [15]; since the loss depends on both, a loss-only fit need not transfer across these interventions. Bergsma et al. report token-based fits with little model-size dependence over their tested range [1]:

$$B_{\text{opt}} \propto D^{0.383}, \qquad B_{\text{crit}} \propto E_{\min}^{0.462}, \tag{78}$$

where $E_{\min}$, as in Eq. (50), is the fewest tokens that reach the loss in question. These findings support measuring on small models, followed by a check at a larger scale; independence from $N$ is an empirical result in those settings, not a theorem for all training recipes. DeepSeek LLM instead fits $B_{\text{opt}}$ against compute [2],

$$B_{\text{opt}} = 0.2920\, C^{0.3271}, \tag{79}$$

with $B$ in tokens and $C = MD$, where $M$ is non-embedding training FLOPs per token, fitted through all batches within 0.25% of the best loss, which fill a broad band. A token-based power law becomes a power law in $C$ along an allocation with $D \propto C^{1-a}$ (Eq. (86)). Its exponent in $C$ is then multiplied by $1 - a$; it is not a universal law across arbitrary model–data splits. DeepSeek notes that the optimum moves slightly when the same compute is split differently. Their batch units and tuning protocols also differ. Treat the exponents as priors for a sweep, rather than directly copying a numerical batch between tokenizations, contexts, or modalities.

**Choose for the resource that matters.** At fixed $N, D$, measure step time $t_{\text{step}}(B)$ on the intended hardware, including communication and gradient accumulation. A practical choice solves, over feasible tested batches,

$$B_{\text{chosen}} \in \arg\min_B \frac{D}{B} t_{\text{step}}(B) \quad \text{subject to an agreed loss or efficiency tolerance.} \tag{80}$$

Use repeated evaluation to decide whether a loss difference is meaningful. If batch is enlarged only by adding accumulation steps, the step count falls but step time rises; wall time need not improve. With more devices, include their cost as well as elapsed time. For a target-loss objective, replace $D/B$ by the measured $S(B;\ell)$ and allow token cost to change.

In practice, search from the measured low-loss region toward the knee, $B_{\text{opt}} \lesssim B \lesssim B_{\text{crit}}$, when the measured quantities have that ordering. At fixed tokens, accept a larger batch only if its loss penalty is acceptable; at fixed target loss, account for the extra tokens it requires. Do not exceed the knee by default: require a measured wall-time benefit that justifies the loss or compute cost. A strict efficiency tolerance may stop the search well below the knee. The definitions do not guarantee an ordering for every recipe. Freeze the batch-selection rule before the final ladder fit. Part IV, Section 4 gives the operational batch-sweep procedure.

## 5.2 Improving critical-batch estimation

The classical measurement fixes a target loss and measures its step–token trade-off. A fixed-budget experiment instead knows the model and token budget before it knows the attainable loss. An absolute loss tolerance answers that practical question, but its meaning depends on the task and loss scale [9, 10].

**Replace a loss gap with equivalent data.** Zheng proposes *equal-loss data efficiency* (ELDE) [16]. Fix $N$, a token budget $E$, and a schedule family. Choose an efficient reference batch $B_{\text{ref}}$. At that batch, train several completed horizons, retuning the selected hyperparameters, and fit

$$\widehat{\mathcal{L}}_{N,B_{\text{ref}}}(E') = \ell_{\infty,N} + A_N (E')^{-\beta_N}. \tag{81}$$

For each candidate $B$, train on $E$ tokens and invert this fit:

$$E^{\text{eq}}(B) = \left( \frac{A_N}{\mathcal{L}_{N,B}(E) - \ell_{\infty,N}} \right)^{1/\beta_N}, \qquad \xi(B) := \frac{E^{\text{eq}}(B)}{E}. \tag{82}$$

Thus $\xi = 0.9$ means the reference reaches the candidate's loss using 90% as many tokens. For a chosen efficiency tolerance $\xi_*$, define

$$B_{\text{ELDE}}(N, E; \xi_*) := \sup\{B : \xi(B) \geq \xi_*\}. \tag{83}$$

Only the reference batch needs a horizon fit; candidates need endpoint measurements. Intermediate checkpoints of an unfinished schedule are not equivalent endpoints. The tolerance remains a user choice, now expressed in data efficiency. This is a proposed estimator, not a universally validated replacement for batch sweeps.

**Relate the threshold to the classical knee.** The conversion is conditional on the equal-loss trade-off of Section 3.4. At $\ell_B := \mathcal{L}_{N,B}(E)$, write $E = E_{\min} + BS_{\min}$ and $E^{\text{eq}} = E_{\min} + B_{\text{ref}} S_{\min}$. Solving these two equations gives

$$B_{\text{crit}}(\ell_B) = \frac{E_{\min}}{S_{\min}} = \frac{\xi(B)B - B_{\text{ref}}}{1 - \xi(B)}. \tag{84}$$

This algebra is also the connection made by Zheng [16]. For $B_{\text{ref}} \ll B$ at a threshold crossing, $B_{\text{ELDE}} \approx (1-\xi_*)B_{\text{crit}}/\xi_*$: a 90% efficiency threshold gives about one ninth of the classical knee, whereas a 50% threshold gives approximately the knee. The knee is evaluated at $\ell_B$, which can change with $B$.

**Check the inversion before trusting the answer.** Use a reference batch whose efficiency is supported across the fitted horizons. Constrain $A_N, \beta_N > 0$. The inverse requires $\mathcal{L}_{N,B}(E) > \ell_{\infty,N}$ and a reliable fit over the inferred horizon. Holding the fit parameters fixed, a small loss error is amplified according to

$$\delta \log E^{\text{eq}} \approx -\frac{\delta \mathcal{L}}{\beta_N(\mathcal{L} - \ell_{\infty,N})}.$$

Uncertainty in the fitted floor adds to this error. Report an interval from repeated runs and refits, and bracket a threshold crossing with measured batches. If the curve is nonmonotone, or $\xi > 1$, inspect reference tuning and measurement noise before interpreting a single largest passing batch. ELDE measures relative data efficiency; hardware timing remains a separate measurement.

### 5.3 Designing a scaling ladder

The purpose of a scaling ladder is to learn about an expensive run while experiments are still cheap: how large should the model be, how long should it train, and what improvement can we expect? A small family of models, trained at several compute budgets with a reasonably tuned recipe, gives us a way to answer these questions before committing to the final run.

**What stays the same as the model grows?** A loss curve is easiest to interpret when its points describe the same task. For an LLM, this means a common data mixture, tokenizer, context length, and evaluation set. For a DiT, the latent representation, patch size, resolution, conditioning, timestep sampling, and loss weighting also define the experiment. Generation quality needs its own comparison with a common sampler and sampling budget. Data reuse matters in both cases: repeatedly seeing a finite dataset is a different regime from continually seeing fresh data.

There are also two different meanings of the training budget. The total available compute $C_{\text{all}}$ pays for the experiments as well as the final run, whose budget we call $C_T$:

$$C_{\text{all}} = C_{\text{pilot}} + C_T + C_{\text{other}}. \tag{85}$$

The pilot budget includes tuning, allocation sweeps, repeat seeds, and held-out validation runs; evaluation and other costs occupy the remaining term. For a dense LLM, our working approximation is $C \approx 6ND$, with a consistent parameter-count convention. For a DiT, it is more convenient to write $C \approx f_{\text{train}}(N)D_{\text{img}}$, where $f_{\text{train}}$ is training FLOPs per image at the chosen representation. MoE and substantial attention overhead call for a more detailed cost model. FLOPs describe the algorithmic budget; GPU-hours also reflect how well each configuration uses the hardware.

**Model size and model shape.** The main size axis can simply be $N$. A reasonable configuration family might roughly double the parameter count between successive sizes, keep FFN ratios and the attention-head rule consistent, and round dimensions for efficient kernels. Width and

depth determine how these sizes are realized. They do not have to become two additional search axes.

Kaplan et al. find weak dependence on Transformer shape at fixed non-embedding parameter count over a broad range, with exceptions for very shallow or extreme shapes [8]. This supports treating $N$ as the main size variable for a first ladder. It does not establish shape independence for DiT, MoE, every optimization recipe, or hardware throughput. A small matched-$N$ comparison can resolve doubts about an unusual family or a sharp change in shape. A broad width–depth grid is better treated as a separate architecture study.

**Finding the best allocation at each rung.** Suppose we choose four or five geometrically spaced compute budgets, $C_i = C_{\max}/4^i$. At each budget, three model sizes around a provisional optimum—say $N_{\text{guess}}/2$, $N_{\text{guess}}$, and $2N_{\text{guess}}$—give a first view of the allocation curve. Each model receives $D = C_i/(6N)$ tokens, so the comparison spends the same compute on a smaller model trained longer or a larger model trained less. The minimum gives $N_i^*$ and $\mathcal{L}_i^*$. A minimum at the boundary calls for a wider sweep; a flat or noisy minimum needs more measurements. These rung counts and spacings are illustrative design choices.

A useful initial guess is $N_{\text{guess}} \propto C^{1/2}$: later LLM allocation studies find exponents near $1/2$, although the optimal tokens per parameter and the precise exponent vary with the setup [6, 3, 2]. The familiar 20 tokens per parameter is therefore a starting guess for some LLM settings, rather than a universal optimum. If deployment already fixes $N$, the question becomes how performance improves with tokens at that size. A single fixed-$D/N$ path can predict that path, but cannot identify the best allocation or separate model and data effects. The same equal-compute comparison applies to DiT, with training images determined by the per-image cost above.

**The training schedule is part of the experiment.** Warmup followed by cosine decay is a straightforward baseline. Each run gets a schedule matched to its own token or image budget, with a common warmup rule and final-to-peak LR ratio. This detail matters: a checkpoint halfway through a long cosine run has not received the cooldown of a finished short run. When the target already has an established schedule, that schedule is a natural starting point for the ladder as well.

A warmup–stable–decay schedule can reuse a stable-training prefix and branch into completed cooldowns at several horizons. This can make the experiment much cheaper: Hägele et al. obtain comparable losses with constant LR plus cooldown while saving substantial compute in their LLM scaling experiments [4]. A small comparison against the chosen baseline tells us whether that saving carries over to our setup. Cosine and WSD may prefer different peak LRs and decay durations.

**Setting the peak learning rate.** A scaling ladder does not require a fresh LR sweep at every point. Chinchilla adopts existing heuristics for the optimization hyperparameters; Llama 3 sets the peak LR between $2 \times 10^{-4}$ and $4 \times 10^{-4}$ according to model size [6, 3]. A mature LR rule is therefore a reasonable starting point, with nearby rates checked at a few representative scales when needed. This differs from explicitly searching and fitting $\eta_{\text{opt}}(C)$, as DeepSeek LLM does [2]: that is an optional study of hyperparameter scaling. Changing batch or training length can still warrant a check of LR and decay, since the number of updates affects accumulated AdamW shrinkage. Part II discusses these interactions.

None of this requires $\mu$P. Empirically tuned rates in an ordinary parameterization are a valid way to construct a scaling study. $\mu$P/CompleteP provides another route: the ladder shares

initialization and tensor-specific scaling rules, while any remaining base-LR drift is measured empirically. The choice belongs to the training recipe used by the experiments.

**What do we actually need to fit?** Once the allocation minima and their recipes have been measured, a small set of curves connects them to the final training decision:

| Fit | Decision it supports |
|---|---|
| $N^*(C) = k_N C^a$ | Select model size; derive the training tokens from compute. |
| $\mathcal{L}^*(C) = \ell_\infty + AC^{-\alpha}$ | Predict final loss and check whether the gain justifies the larger run. |
| $B_{\text{opt}}(C) = k_B C^b$ | Locate a good batch region along the fitted allocation path. |

The first fit already determines

$$D^*(C) = \frac{C^{1-a}}{6k_N}, \qquad \frac{D^*(C)}{N^*(C)} = \frac{C^{1-2a}}{6k_N^2}. \tag{86}$$

An independent token-allocation fit therefore adds no degree of freedom under the same cost approximation. A joint $\mathcal{L}(N, D)$ fit becomes useful when the experiment covers enough model–data combinations to identify both effects. Batch may be better predicted by $D$ than by $C$ when comparing different allocations. Measuring $B_{\text{crit}}$ or an ELDE threshold adds the information needed to trade some efficiency for a faster, larger batch.

These curves summarize one recipe and allocation path. Their exponents need to be measured, and a sparse ladder can leave even the loss floor poorly determined. Sensitivity to the fitting range and floor is therefore part of the result, alongside the fitted coefficients.

**How close does the ladder need to get to the final run?** A natural question is whether the largest pilot should use $C_T/10$, $C_T/5$, or even $C_T/2$. These choices change the distance of the extrapolation; none comes with a general confidence guarantee:

| Largest pilot $C_{\max}$ | Target/pilot compute | What it buys |
|---|---|---|
| $C_T/10$ | 10× | Lower pilot cost; a longer extrapolation |
| $C_T/5$ | 5× | A closer validation point |
| $C_T/2$ | 2× | Shorter extrapolation; an expensive pilot |

These fractions refer to *one run versus the final run*, not the sum of all experiments versus the total available budget. With $m$ full-run equivalents per compute rung and rung ratio four,

$$C_{\text{pilot}} = mC_{\max} \sum_{i=0}^{K-1} 4^{-i} < \frac{4m}{3} C_{\max}. \tag{87}$$

For example, four rungs with three allocation candidates each cost $3C_{\max}(1+1/4+1/16+1/64) \approx 3.98C_{\max}$, even before extra tuning. If $C_{\max} = C_T/10$, these runs already cost about $0.40C_T$, or 28.5% of the combined pilot-plus-final budget. If only 10% of that combined budget is reserved for these pilots, their largest run is about $C_T/36$. This is why the largest pilot and the total experimental budget answer different planning questions.

One possible planning starting point is a largest validation run near $C_T/10$, provided the smaller sweeps and tuning also fit within the pilot budget. The evidence for trusting this choice comes from prediction. We fit the smaller runs first, predict a larger held-out run, and then compare the prediction with its measured loss and allocation optimum. Ideally, this backtest spans a compute ratio similar to the intended final extrapolation. Nearby LR and batch settings reveal whether a miss comes from an inadequate recipe; repeat seeds and leave-one-rung-out fits reveal how sensitive the curve is to its measurements. A useful prediction error is one small enough relative to the expected gain from scaling to support the decision we are making.

A drifting fit, a missed held-out result, or an optimum outside the measured bracket gives us a reason to expand the experiment. A rung near $C_T/5$ may then be worth its cost, although that cost reduces the compute left for the final run. Even a rung at $C_T/2$ cannot establish transfer to a changed data regime or training recipe. Statistical fit intervals describe uncertainty within the assumed model; held-out predictions help us assess whether that model is useful at larger scales.

**From the fitted curves to a training run.** The remaining budget $C_T$ now gives a predicted $N_T$. Rounding this to a realizable configuration determines $D_T = C_T/(6N_T)$. The fitted $B_{\text{opt}}$ supplies a starting batch; the measurements in Sections 5.1–5.2 tell us whether a larger $B_T$ offers an attractive time–quality trade-off. The step count then follows:

$$S_T \approx \frac{D_T}{B_T} = \frac{C_T}{6N_T B_T}. \tag{88}$$

Here $B_T$ is global tokens per optimizer step. For DiT, with global image batch $B_{\text{img}}$, use $S_T \approx C_T/[f_{\text{train}}(N_T)B_{\text{img}}]$. Changing batch changes the update count even when total training compute stays fixed.

The final run inherits the initialization, LR, residual factors, and decay rules tested by the pilots, with its schedule matched to this horizon. Part II explains how $\mu$P/CompleteP can supply these scaling rules. If we choose that route, it already shapes the pilot experiments: changing parameterization after fitting would introduce a new recipe whose behavior has yet to be checked. This connects the empirical allocation of Part I to the training dynamics of Part II:

A common task, model family, and training recipe
↓
Small-run tuning and model–data allocation curves
↓
Fits for allocation, loss, and batch
↓
Prediction checked against a larger held-out run
↓
The final $N_T, D_T, B_T, S_T$, using the validated recipe

## References

[1] Shane Bergsma, Nolan Dey, Gurpreet Gosal, Gavia Gray, Daria Soboleva, and Joel Hestness. Power lines: Scaling laws for weight decay and batch size in LLM pre-training. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL: https://arxiv.org/abs/2505.13738.

[2] Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, et al. DeepSeek LLM: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954*, 2024. URL: https://arxiv.org/abs/2401.02954.

[3] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL: https://arxiv.org/abs/2407.21783.

[4] Alexander Hägele, Elie Bakouch, Atli Kosson, Loubna Ben Allal, Leandro Von Werra, and Martin Jaggi. Scaling laws and compute-optimal training beyond fixed training durations. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL: https://arxiv.org/abs/2405.18392.

[5] Joel Hestness, Sharan Narang, Newsha Ardalani, Gregory Diamos, Heewoo Jun, Hassan Kianinejad, Md. Mostofa Ali Patwary, Yang Yang, and Yanqi Zhou. Deep learning scaling is predictable, empirically. *arXiv preprint arXiv:1712.00409*, 2017. URL: https://arxiv.org/abs/1712.00409.

[6] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL: https://arxiv.org/abs/2203.15556.

[7] Hongzhi Huang, Defa Zhu, Banggu Wu, Yutao Zeng, Ya Wang, Qiyang Min, and Xun Zhou. Over-tokenized transformer: Vocabulary is generally worth scaling. *arXiv preprint arXiv:2501.16975*, 2025. URL: https://arxiv.org/abs/2501.16975.

[8] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. URL: https://arxiv.org/abs/2001.08361, doi:10.48550/arXiv.2001.08361.

[9] Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. An empirical model of large-batch training. *arXiv preprint arXiv:1812.06162*, 2018. URL: https://arxiv.org/abs/1812.06162, doi:10.48550/arXiv.1812.06162.

[10] William Merrill, Shane Arora, Dirk Groeneveld, and Hannaneh Hajishirzi. Critical batch size revisited: A simple empirical approach to large-batch language model training. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL: https://arxiv.org/abs/2505.23971.

[11] Tim Pearce and Jinyeop Song. Reconciling Kaplan and Chinchilla scaling laws. *Transactions on Machine Learning Research*, 2024. URL: https://arxiv.org/abs/2406.12907.

[12] Tomer Porian, Mitchell Wortsman, Jenia Jitsev, Ludwig Schmidt, and Yair Carmon. Resolving discrepancies in compute-optimal scaling of language models. *arXiv preprint arXiv:2406.19146*, 2024. URL: https://arxiv.org/abs/2406.19146.

[13] Jonathan S. Rosenfeld, Amir Rosenfeld, Yonatan Belinkov, and Nir Shavit. A constructive prediction of the generalization error across scales. In *International Conference on Learning Representations*, 2020. URL: https://arxiv.org/abs/1909.12673.

[14] Lili Yu et al. Scaling embedding layers in language models. *arXiv preprint arXiv:2502.01637*, 2025. URL: https://arxiv.org/abs/2502.01637.

[15] Hanlin Zhang, Depen Morwani, Nikhil Vyas, Jingfeng Wu, Difan Zou, Udaya Ghai, Dean Foster, and Sham Kakade. How does critical batch size scale in pre-training? In *International Conference on Learning Representations*, 2025. URL: https://arxiv.org/abs/2410.21676.

[16] Chenyu Zheng. ELDE: A practical method for estimating critical batch size. Author's blog, August 20, 2026. In Chinese; title translated. Accessed October 20, 2026. URL: https://chen-yu-zheng.github.io/blog/2026/CBS03/.